

Introduzione alla Autenticazione come Servizio di Terza Parte

Kerberos

Virtualizzazione e Integrazione di sistemi

Contenuto del documento

- Preliminari su Crittografia
- Introduzione a Kerberos
- Kerberos - Protocolli di Fase 1, 2 e 3
- Considerazioni

Crittografia - elenco argomenti

- Codifica e Decodifica.
 - Crittografia a chiave segreta (simmetrica)
 - DES, Triple DES, AES.
 - Crittografia a chiave pubblica (asimmetrica)
 - RSA
- Funzioni Hash crittografiche.
 - MD5 , SHA-1
- Applicazioni
 - Tecnica MAC per Integrità del messaggio
 - Firma Digitale
- ~~Certificazione della chiave pubblica.~~

Crittografia

- Scienza antichissima: codificare e decodificare informazione. Tracce risalenti all'epoca di Sparta
- Codificare: testo in chiaro $\rightarrow$ testo codificato
- Decodificare: testo codificato $\rightarrow$ testo in chiaro
- Ambedue basate su: **algoritmo** e **chiave**
 - Es: “Shiftare” di k una stringa
- Algoritmo pubblico!
- Sicurezza data da:
 1. segretezza della chiave
 2. robustezza dell'algoritmo

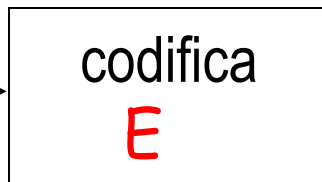
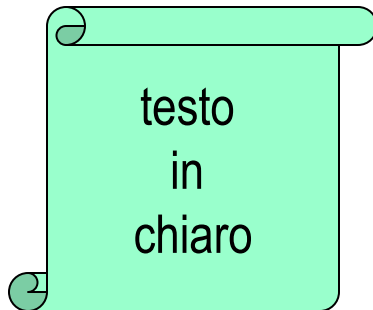
Codifica e decodifica

Le funzioni di codifica e decodifica sono noti,
la chiave di cifratura no

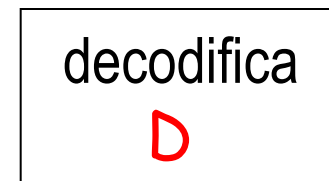
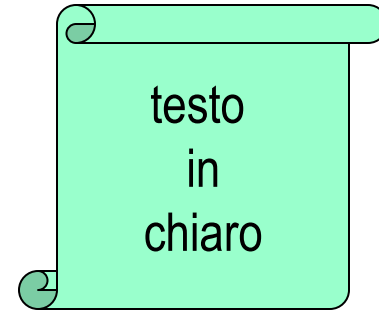
E_{K1} = encoding con chiave K1
 D_{K2} = decoding con chiave K2
t.c $D_{K2} (E_{K1} (\text{testo})) == \text{testo}$

Mittente

Destinatario



2



testo cifrato

Chiave **K1** di
codifica e decodifica

Chiave **K2** di
codifica e decodifica

Crittografia a chiave segreta (simmetrica)

Le funzioni di codifica e decodifica sono note, la chiave è nota solo ai 2 che comunicano

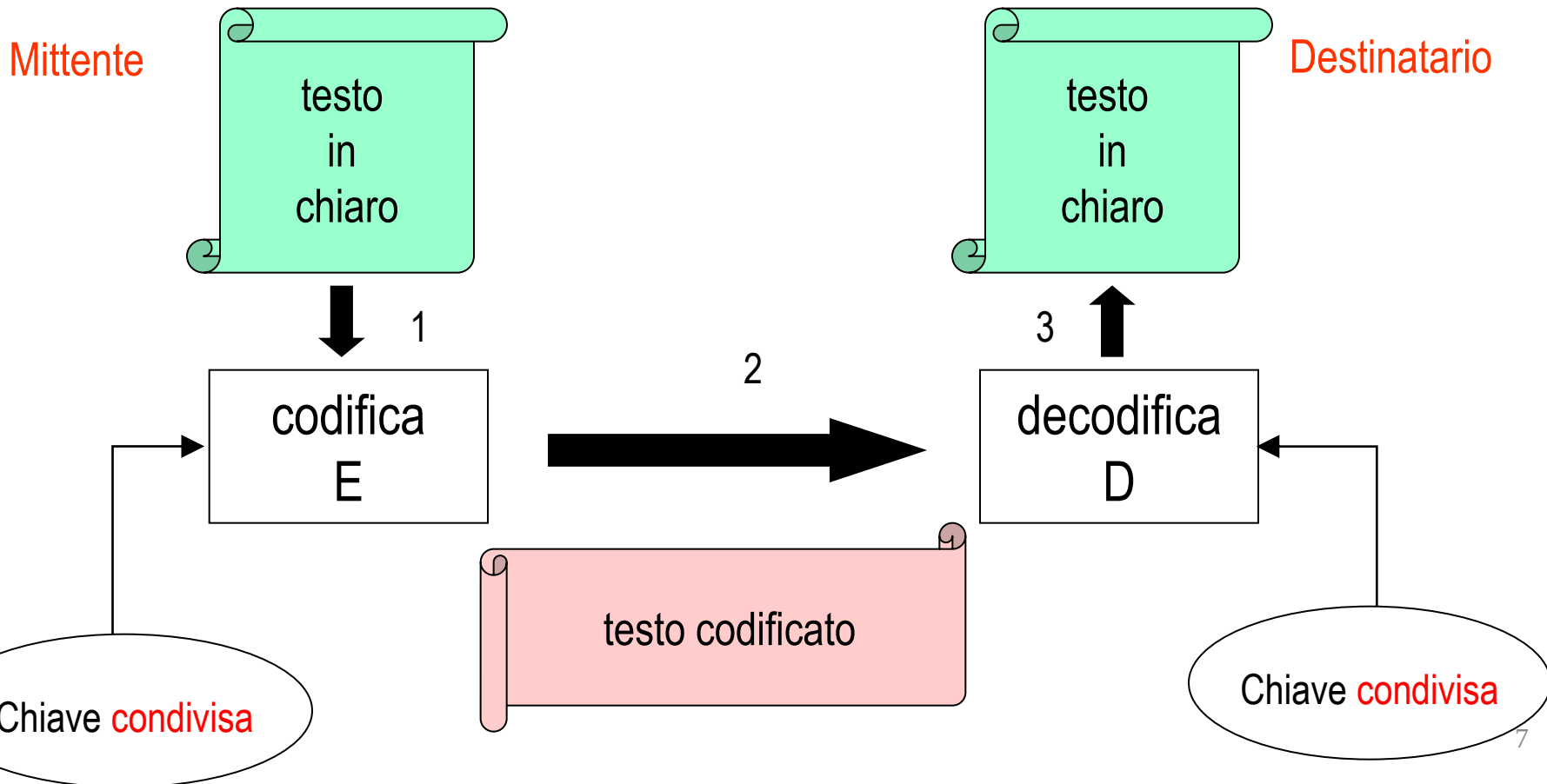
- Medesima chiave per codifica e decodifica.
 - Richiede che solo mittente e destinatario conoscano la chiave segreta.
 - Segretezza, autenticazione, integrità, ma non "Non repudiabilità".
 - Proprietà dipendono dalla segretezza della chiave (che è nota solo a 2).
-
- + Di solito DES (Data Encryption Standard), **Triple-DES**, **AES** (Advanced Encryption Standard, noto come Algoritmo di Rijndael) e altri.
 - + DES non più sicuro.
 - + usano chiavi di varie dimensioni 64,128,168, 256 bit.
 - + al crescere della dimensione aumenta la sicurezza.
 - + abbastanza veloci.
-
- Occorre distribuire una chiave a ciascuna coppia di utenti.
 - Per n utenti servono $n*(n-1)/2$ chiavi diverse

Proprietà codifica a chiave segreta

Segretezza, Autenticazione, Integrità

E_K = encoding con chiave K
 D_K = decoding con chiave K
t.c $D_K (E_K (\text{testo})) == \text{testo}$

manca Non
Repudiabilità



Pro e contro della Crittografia a chiave segreta

PRO

- Gli algoritmi (Triplo DES, AES e altri) possono essere implementati in modo efficiente

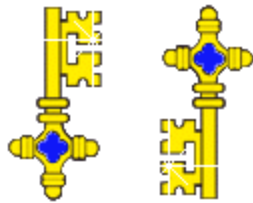
CONTRO

- Le parti che comunicano devono prima scambiarsi la chiave in modo sicuro
- Questo è un punto critico per il quale non esistono al momento soluzioni generali affidabili

Crittografia a chiave pubblica (asimmetrica)

**Le funzioni di codifica/decodifica sono note,
la chiave pubblica è nota, la chiave privata è segreta**

- Una chiave per codifica, un'altra per decodifica
- Mittente e destinatario NON condividono una chiave segreta.
- **Ogni utente ha una coppia di chiavi**
 - **chiave privata**: segreto personale da custodire
 - **chiave pubblica**: informazione da diffondere a tutti
- **Entrambe usabili per codificare o decodificare**, e soprattutto
- **il testo codificato con una chiave può essere decodificato solo con l'altra chiave.**
- Di solito algoritmo **RSA** (*dagli autori Rivest, Shamir e Adelson*)
- Usano chiavi di 1024-2048 bit
- Sotto 2048 non consigliabili
- Sono un po' lenti
- + Segretezza, autenticazione, integrità...



Crittografia asimmetrica

Proprietà funzioni crittografiche e chiavi

La funzione di cifratura è denotata con il simbolo K seguito da un apice $+$ o $-$ a indicare rispettivamente cifratura con chiave pubblica o privata.

Indichiamo con $K^+(\text{testo})$ la cifratura di un testo mediante chiave pubblica di un utente.

Indichiamo con $K^-(\text{testo})$ la cifratura mediante chiave privata di un utente.

Allora

$$K^+ (K^- (\text{testo})) == \text{testo}$$

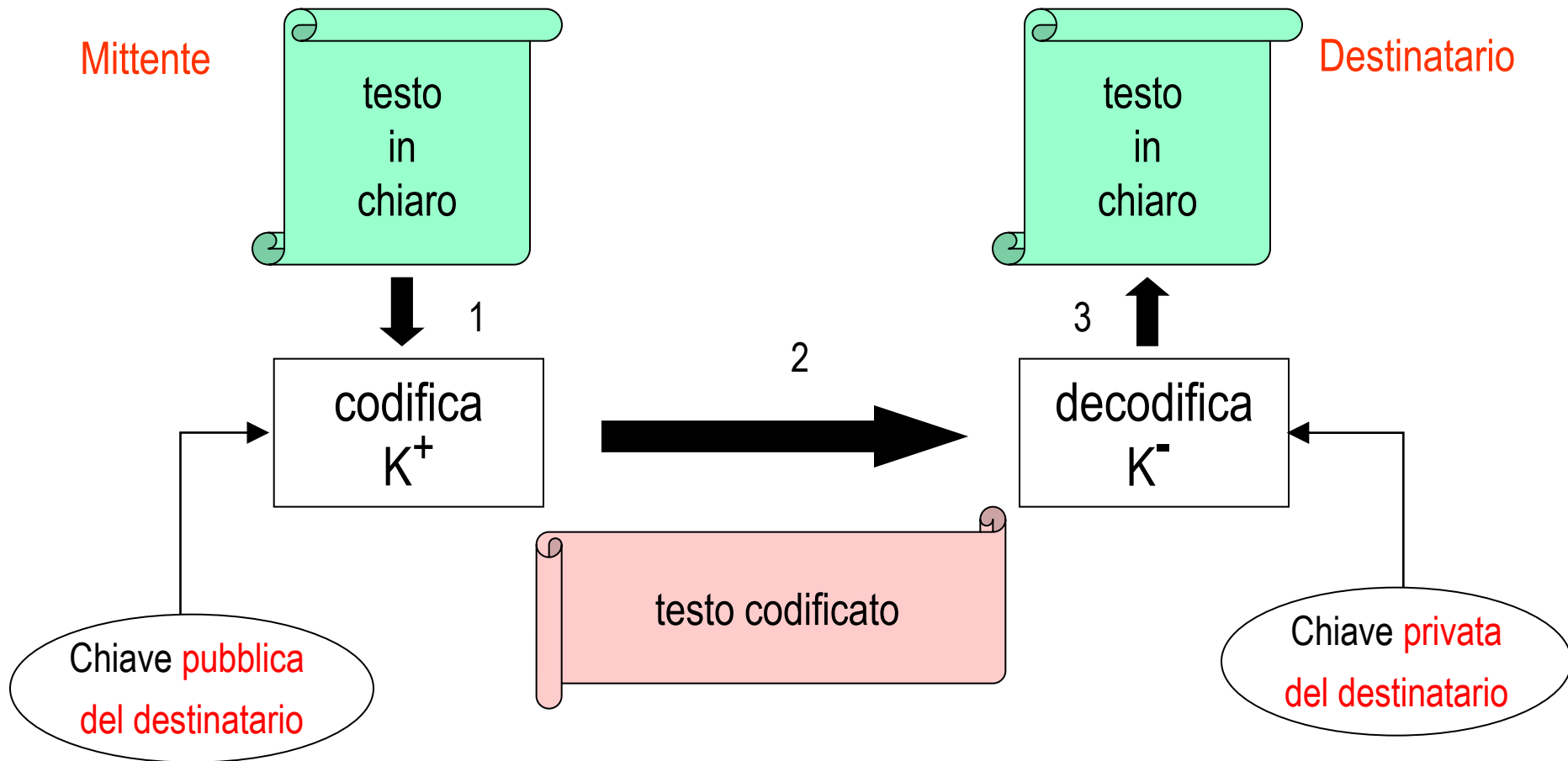
$$K^- (K^+ (\text{testo})) == \text{testo}$$

Dalla chiave pubblica K^+ , deve essere impossibile calcolare la chiave privata K^- .

Dal testo codificato deve essere impossibile trovare il testo in chiaro senza conoscere la chiave.

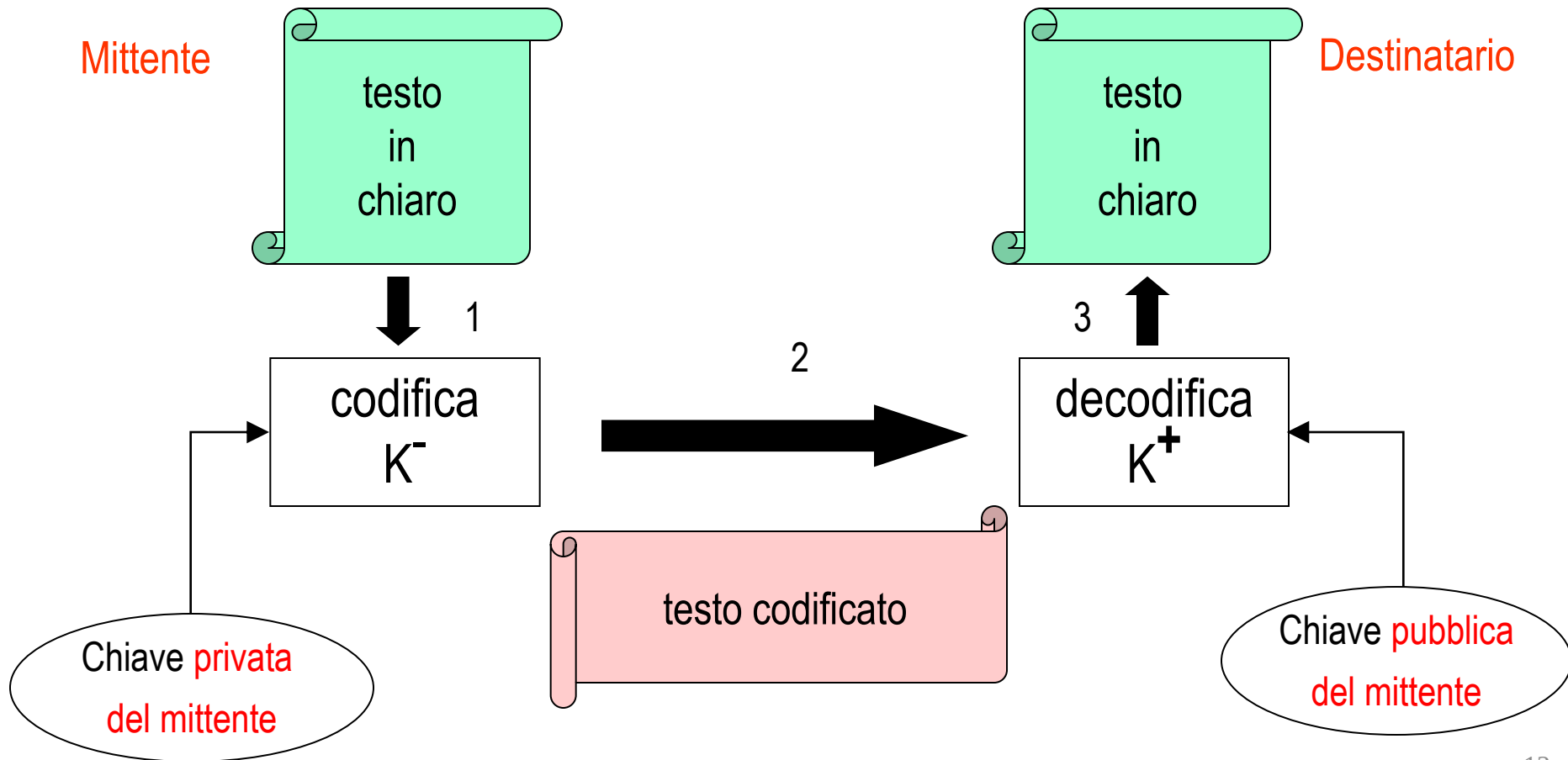
Proprietà codifica a chiavi asimmetriche

Segretezza



Proprietà codifica a chiavi asimmetriche

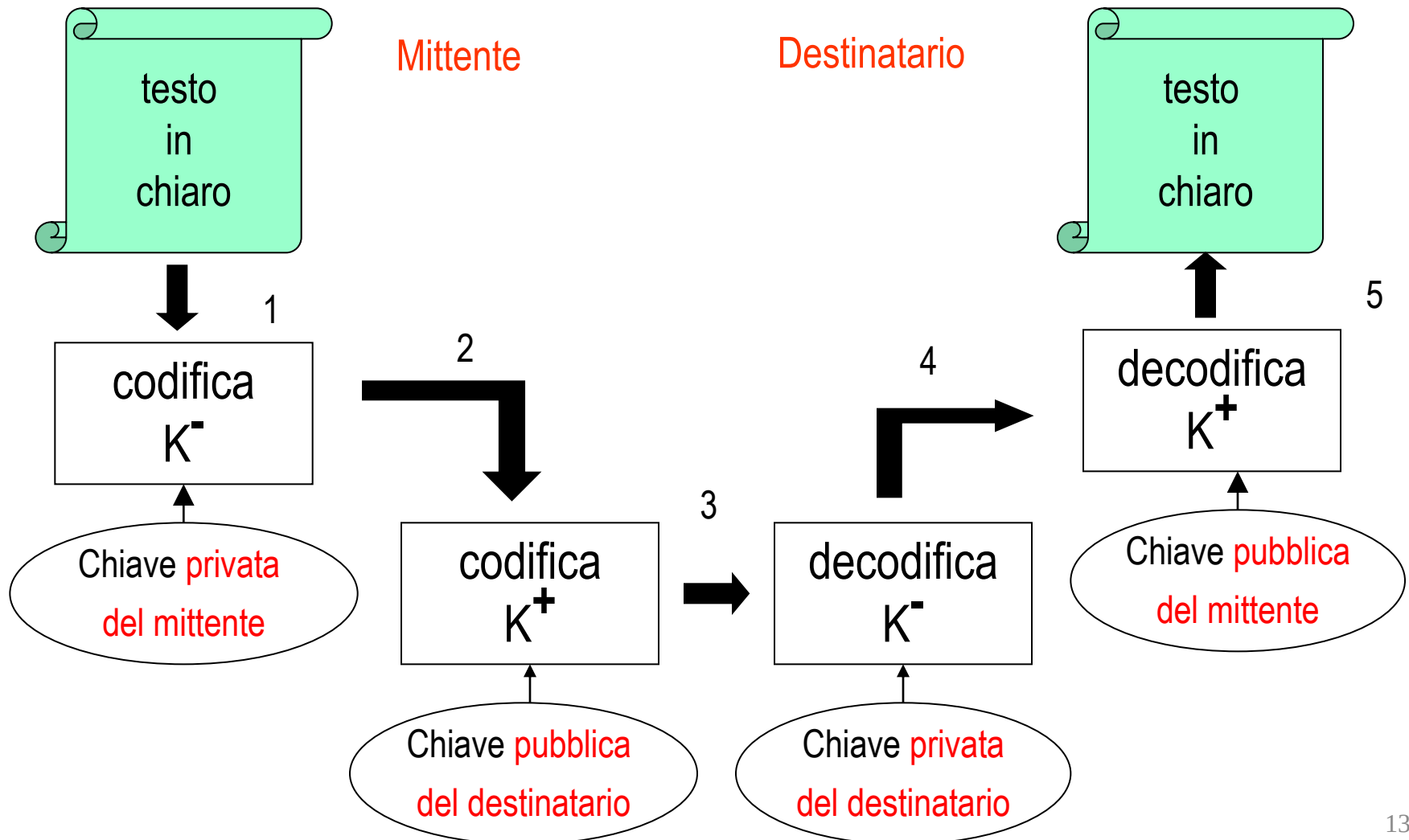
Autenticazione, integrità e Non Repudiabilità



Proprietà codifica a chiavi asimmetriche

Le quattro insieme

Segretezza, Autenticazione, integrità e Non Repudiabilità



Pro e contro della Crittografia a chiave pubblica

PRO

- Non occorre scambiarsi chiavi segrete: le uniche chiavi che è necessario comunicare sono quelle pubbliche (che per definizione sono pubbliche!)

CONTRO

- Gli algoritmi di crittografia asimmetrica sono più lenti di quelli per crittografia simmetrica
- Come facciamo ad essere certi che la chiave pubblica, ad es., di Alice è *veramente di Alice*?

Funzioni Hash Crittografiche

- Sono anche dette **one-way hash** o anche **funzioni digest** cioè funzioni riassunto.

Def: una **Funzione hash crittografica** $MD()$, message digest, è una funzione che:

1. prende in input una stringa (**messaggio**) m , a lunghezza variabile
2. e produce una stringa (**digest**) a lunghezza fissa $MD(m)$ che possiamo pensare come un riassunto digitale del messaggio.
3. e che soddisfa la seguente proprietà di **Non reversibilità**
 - ❑ Deve essere computazionalmente impossibile trovare due messaggi x e y tali che $MD(x) = MD(y)$
 - Tale proprietà può essere espressa anche così:
 - dato $y = MD(x)$, con x sconosciuta, allora è impossibile determinare x .
- ~~4. Se due messaggi m_1 e m_2 differiscono di poco, i due digest dovrebbero essere assai diversi.
 - E' però impossibile garantire che 2 messaggi diversi abbiano sempre digest diversi.~~

Differenze

- ❑ Le funzioni **Hash crittografiche** differiscono dalle funzioni di **codifica e decodifica** in quanto
 - i. le hash non consentono una decodifica, cioè non sono invertibili.
 - ii. inoltre le Hash non prevedono l'uso di una chiave.
 - iii. Infine, le funzioni di codifica e decodifica non restituiscono messaggi a lunghezza fissa.
- ❑ Le funzioni **Hash crittografiche** differiscono dal **checksum di Internet** poiché
 - i. le checksum Internet soddisfano le proprietà 1 e 2 ma non la 3. Le checksum sono funzioni hash ma non sono crittografiche.

Funzioni Hash crittografiche più importanti

~~MD5 (Message Digest 5)~~

- ~~• Sviluppato da Ronald Rivest nel 1991~~
- ~~• Genera un digest lungo 128 bit~~
- ~~• Oggi non è ritenuto abbastanza sicuro~~

SHA (Secure Hash Algorithm)

- Famiglia di funzioni hash crittografiche sviluppate dalla NSA a partire dal 1993.
- **SHA-1** genera un digest lungo 160 bit ma è **poco sicuro**.
- **SHA-2** indica genericamente le successive versioni, che si distinguono per numero di bit del digest.
 - SHA-224, SHA-256, SHA-384, SHA-512
 - SHA-x genera un digest lungo x bit.
 - Ad esempio, SHA-512 genera un digest lungo 512 bit.
 - La sicurezza e (purtroppo) il tempo necessario a calcolare il digest, crescono con la dimensione del digest.
 - Si preferisce usare SHA-256 o **SHA-512**

Integrità di messaggi e Firma Digitale

MAC - Message Authentication Code

- ❑ La tecnica detta **Message Authentication Code** consente di
 - ❑ **autenticare** (parzialmente) l'**origine** di un messaggio (riconoscere il mittente)
 - ❑ e **garantire l'integrità** del messaggio stesso
- ❑ facendo uso combinato della crittografia (a chiave pubblica o a chiave segreta) e delle funzioni hash crittografiche.
- ❑ Se tra mittente e destinatario esiste una chiave segreta per crittografia simmetrica,
 - ❑ allora si può usare funzione Hash e crittografia simmetrica.
 - ❑ Questa è la tecnica **MAC** vera e propria.
- ❑ Se invece il mittente è dotato di propria chiave pubblica (coppia di chiavi pubblica e privata),
 - ❑ allora si può usare funzione Hash e crittografia asimmetrica.
 - ❑ Questa è la tecnica di **Firma Digitale**.

MAC -Message Authentication Code

Spedizione

- ❑ Supponiamo di dover spedire un messaggio m .
- ❑ La tecnica della Firma Digitale del messaggio mediante MAC, stabilisce di eseguire le seguenti operazioni:

In spedizione del messaggio m , occorre

- ❑ applicare al messaggio una Funzione Hash Crittografica per creare una stringa che sia un riassunto (digest) del messaggio stesso. Otteniamo $MD(m)$
- ❑ Codificare il riassunto (digest) del messaggio
 - ❑ o con la codifica a chiave segreta, usando la chiave segreta K condivisa tra mittente e destinatario. Otteniamo $MAC = E_K(MD(m))$
 - ❑ oppure con la codifica a chiave pubblica, utilizzando la chiave privata del mittente. Otteniamo $MAC = K^-(MD(m))$
- ❑ Il riassunto così codificato viene detto Codice di Autenticazione del Messaggio (Message Authentication Code, MAC).
- ❑ Il MAC del messaggio viene concatenato al messaggio stesso.
- ❑ Viene spedito il nuovo messaggio formato dal messaggio unito al suo MAC (m, MAC) cioè $(m, K^-(MD(m)))$ oppure $(m, E_K(MD(m)))$.

MAC -Message Autentication Code

Ricezione

Alla ricezione del messaggio con il MAC (m' , MAC), occorre:

- ❑ separare il messaggio m' in chiaro dal MAC del messaggio.
- ❑ Decodificare il MAC
 - ❑ o con la codifica a chiave pubblica, utilizzando la chiave pubblica del mittente. Ottengo $K^+(K^-(MD(m)))=MD(m)$.
 - ❑ oppure con la codifica a chiave segreta utilizzando la chiave segreta condivisa tra mittente e destinatario. Ottengo $D_k(E_k(MD(m)))=MD(m)$.
- ❑ Se la decodifica produce errori significa che il mittente non è quello, dichiarato nel messaggio, di cui conosciamo la chiave. **Fine.**
- ❑ Invece, se la decodifica va a buon fine, quello che otteniamo è il digest del messaggio originale **MD(m)**.
- ❑ Ora calcoliamo il digest del messaggio m' ricevuto **MD(m')**
- ❑ Infine confrontiamo i due digest,
 - ❑ quello ricavato decodificando il MAC **MD(m)**
 - ❑ e quello ricalcolato dal messaggio in chiaro ricevuto **MD(m')**
- ❑ Se i due digest sono uguali **MD(m)=MD(m')** allora il messaggio ricevuto m' è l'originale ed è stato mandato dal mittente dichiarato.

Codice di autenticazione dei messaggi - MAC

usa Chiave Segreta

chiave S
segreta
condivisa



(messaggio)

Alice
MITTENTE

m

$MD(\cdot)$

$MD(m)$

$E(\cdot)$

$E(MD(m))$

accorpa

m $E(MD(m))$

possibili modifiche
al messaggio
ad es: $m \rightarrow m'$

Internet

m' $E(MD(m))$

separa

$E(MD(m))$

$D(\cdot)$

$MD(m)$

consegna

m'

confronta

$MD(m')$

$MD(\cdot)$

m'

$\langle \rangle$
scarta m'

Bob
DESTINATARIO

Firma Digitale - usa Chiave Pubblica

chiave K^-
privata
di Alice



(messaggio)

Alice
MITTENTE

m

$MD(\cdot)$

$MD(m)$

$K^-(\cdot)$

$K^-(MD(m))$

accorpa

m

$K^-(MD(m))$

possibili modifiche
al messaggio
ad es: $m \rightarrow m'$

Internet

K^+
chiave
pubblica
di Alice

$MD(m)$

$K^+(\cdot)$

$K^-(MD(m))$

separa

m'

$K^-(MD(m))$



Bob
DESTINATARIO

consegna

m'

confronta

$MD(m')$

$MD(\cdot)$

m'

$\langle \rangle$
scarta m'

Differenze tra MAC, Firma Digitale e Codifica/Decodifica

- La tecnica **MAC** è applicabile solo se mittente e destinatario conoscono una chiave segreta.
- Quindi il destinatario sa che, se riesce a decodificare il messaggio, questo proviene dall'altro (o da sé stesso).
- Però, il destinatario non può dimostrare, ad osservatori esterni, che l'altro ha spedito il messaggio, potrebbe essere stato lui stesso.

=> **Integrità, autenticazione, mancano segretezza e non repudiabilità.**

- La **firma digitale** è applicabile solo se il mittente ha una chiave pubblica/privata.
- Chiunque può leggere il messaggio e riconoscere chi è stato a spedirlo e verificarne l'integrità.

⇒ **Integrità, autenticazione, non repudiabilità, manca segretezza**, c'è broadcast.

- MAC e Firma Digitale sono **più rapide della codifica del documento intero** poiché **NON DEVONO** codificare tutto il documento ma solo il digest.

Domanda a risposta differita

- Con la firma digitale prima descritta non garantisco segretezza.
- Come posso modificare la tecnica per garantire segretezza?
- Perdo qualche proprietà?
- Quanto è pesante computazionalmente la tecnica modificata?
- Ragionateci per casa.

buon divertimento 😊

Kerberos

il controllore delle porte dell'inferno



Kerberos – le origini

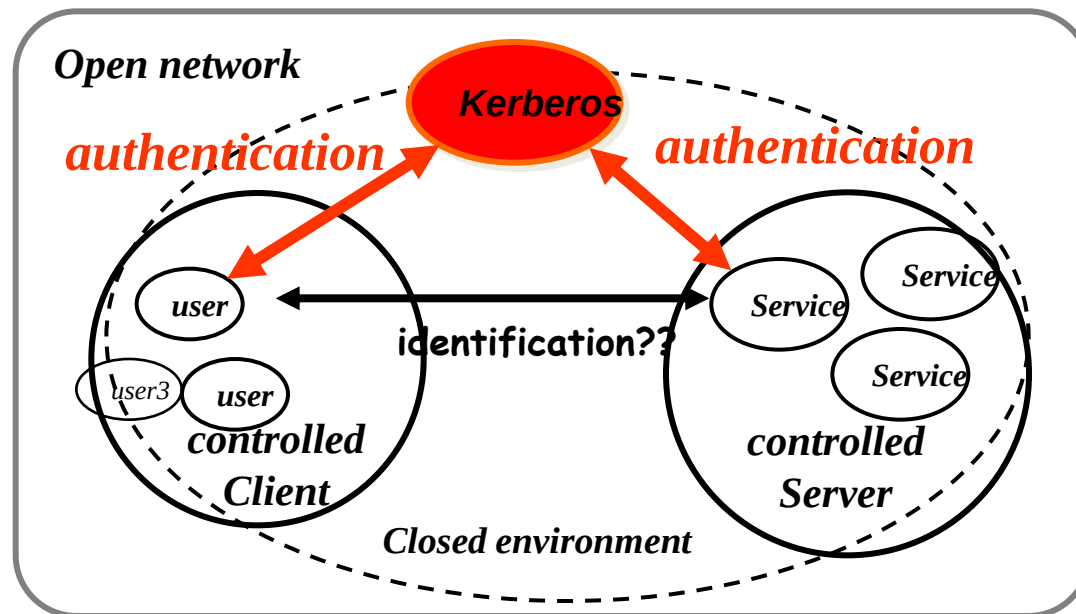
- Sistema di autenticazione nato 1983 dalla collaborazione tra **M.I.T.**(Massachusetts Institute of Technology) **IBM** e **DEC**
- Il nome KERBEROS deriva dalla mitologia greca.
- Era il nome del cane a tre teste a guardia delle porte dell'inferno
- Il cane simboleggia con le sue tre teste i tre scopi originali di KERBEROS ossia
 - **Autenticazione:** verificare l'identità di un client o di un servizio;
 - **Autorizzazione:** autorizzare un client autenticato ad utilizzare un particolare servizio;
 - **Cifratura:** capacità del sistema di prevenire che terze parti ascoltino i contenuti di qualunque comunicazione



- Attualmente sono usate due versioni diverse di Kerberos, che usano protocolli diversi e non compatibili tra di loro:
- versione 4, fine anni '80.
- versione 5, emanata nella RFC 1510 del 1993 e stabilizzata nella RFC 4120 del 2005, più sicura e multidominio.²⁶

Motivation

- Come controllare l'accesso di utenti che richiedono servizi da reti diverse



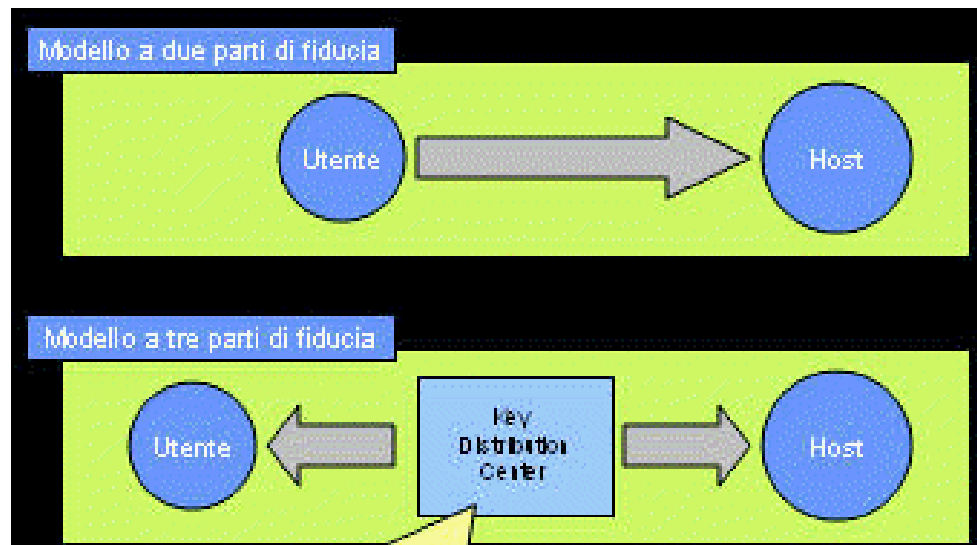
- Requisiti di Autenticazione in open network
 - Secure (?)
 - Reliable (?)
 - Scalable (??)
 - Transparent (???????)

Kerberos - Obiettivi

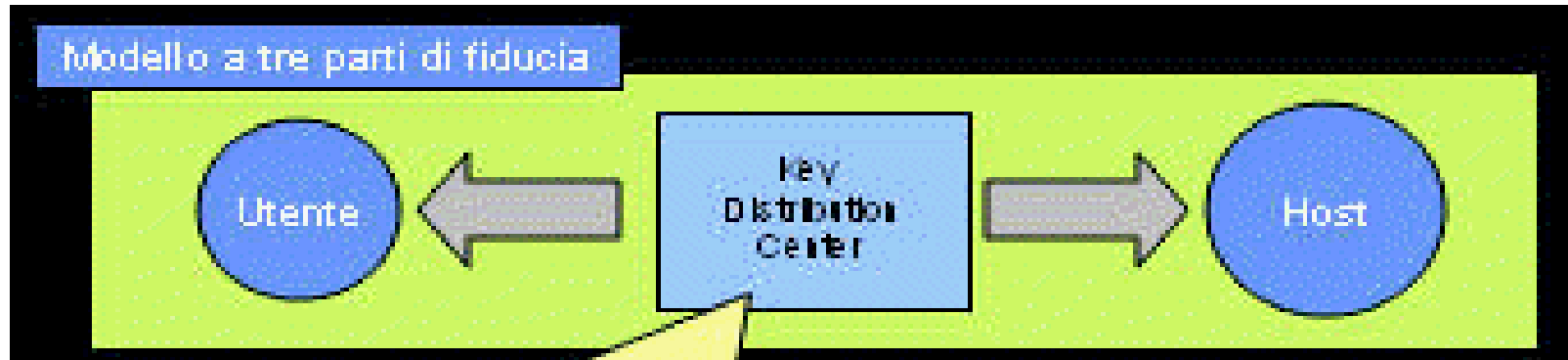
- Server fidato di gestione di chiavi (Trusted key server system)
- Fornisce un servizio di autenticazione centralizzato basato su chiavi private (simmetriche), separato dal servizio stesso (centralised private-key third-party authentication in a distributed network).
- Permette agli utenti di accedere a servizi distribuiti nella rete.
- **senza che l'utente debba fidarsi dei server fornitori di servizio.**
- Ipoteticamente, non richiede agli utenti di fidarsi delle workstation nella rete in cui opera (in realtà non è del tutto vero, deve fidarsi della workstation dell'utente che deve essere sicura **(no keylogger o software simili)**).
- Invece deve fidarsi del server di autenticazione centralizzato.

KERBEROS – modello di Fiducia

- L'autenticazione nel sistema Kerberos si basa su un nuovo modello di fiducia. (a chiavi private con terza parte fidata)
- A differenza del modello a due parti, in cui è prevista la presenza di due elementi aventi fiducia reciproca , ...
- Nel sistema Kerberos le due parti vengono a trovarsi in una relazione di fiducia **SOLO** verso una terza parte avente funzione di garante dell'identità dell'uno verso l'altro.



KERBEROS - Modello a "terza parte di fiducia"



La terza parte è chiamata **KDC** (Key Distribution Center) e si occupa di un dominio detto Realm.

I principal di un Realm sono sia utenti che servizi che si possono autenticare tramite Kerberos.

struttura nome di un principal: **Primary/instance@NOMEREALM**

Primary = equivale al nome dell'utente o del servizio

instance = equivale alla qualifica di un utente.

- ciascun client e server condivide con il KDC una key (o password) crittografica
- KDC si occupa di presentare un principal ad un altro principal tramite le key mantenendone la segretezza reciproca



Le assunzioni di Kerberos

- Le password non sono "facili".
- Le workstation sono sicure (no keylogger o software simili).
- La rete è insicura
 - si riferisce alla rete che collega client e server a kerberos.
 - Kerberos dovrebbe sopportare ogni tipo di attacco, ma qualche minimo rischio esiste, quindi:
 - In the Kerberos architecture, every realm (security domain) must operate a physically secure environment that hosts a key distribution center (KDC).
 - Il significato è che **sarebbe meglio usare kerberos solo all'interno di un dominio controllato.**



Le strategie di Kerberos

- Single sign-on (**la password è richiesta ed usata solo una volta**, per la sequenza di login).
- La password non è **mai trasmessa in chiaro**. (i.e., the system is resistant against password sniffing attacks)
- Si usano **solo chiavi private** (cioè symmetric encoding-decoding) tra i due che devono comunicare (**A pure private-key scheme**, No use of public key cryptography).
- Ogni dominio di sicurezza di kerberos (realm) **deve far eseguire in un ambiente fisico sicuro il servizio di distribuzione delle chiavi** (key distribution center -KDC).
- Implementa un protocollo di autenticazione che estende il protocollo di Needham-Schroeder per la distribuzione di chiavi.
- Il centro di distribuzione delle chiavi (key distribution center -KDC) **mantiene una chiave segreta (secret key) per ciascun principal** (utente, servizio, host) che deve comunicare.

Requisiti - il tempo in Kerberos

- La sicurezza di Kerberos è basata anche sui time stamp (marche temporali) dei ticket perciò è d'importanza critica che gli orologi dei server kerberos siano regolati con accuratezza.
- In kerberos, i ticket hanno una scadenza breve per prevenire attacchi di forza bruta e attacchi di replica.
- Permettendo agli orologi di subire scostamenti si rende la rete vulnerabile a questi attacchi.
- A causa dell'importanza della sincronia degli orologi nella sicurezza del protocollo Kerberos, se gli orologi non sono sincronizzati entro un ragionevole intervallo Kerberos presenta errori fatali e smette di funzionare.



Applicabilità di Kerberos

- Kerberos non è facilmente applicabile in tutti i contesti perché
 - utilizza uno specifico protocollo di comunicazione tra l'entità (applicazione) che richiede un servizio e l'entità che deve fornire il servizio.
- Le applicazioni devono essere modificate per usare il protocollo **kerberos** (kerberizzate - **kerberized**).
- Alcune applicazioni che già supportano kerberos:
 - **AFS, NFS, Coda File System** (distributed networked file system)
 - **Apache** (mod_auth_kerb) e **Apache 2** (libapache-mod-auth-kerb)
 - Cisco **routers** and **switches** running
 - **Eudora, Mulberry** (e-mail client)
 - **Mac OS X**
 - **Microsoft Windows** e successivi (default authentication protocol)
 - **OpenSSH** SSH protocol (Kerberos v5)
 - **PAM** Pluggable authentication modules (pam_krb5 module)
 - **Samba, Netatalk** networking protocol
 - **SOCKS** Internet protocol
 - **X Window System** standard toolkit and protocol to build GUI
 - Qualsiasi software che usi il framework **SASL** (**OpenLDAP, Dovecot IMAP4 , POP3, Postfix**)
 - La suite di Kerberos contiene versioni kerberizzate di **rsh, FTP e Telnet**
 - Qualsiasi software **java** che utilizzi JAAS/JGSS
 - È possibile modificare qualsiasi applicazione affinché utilizzi Kerberos

Why Kerberos and LDAP together?

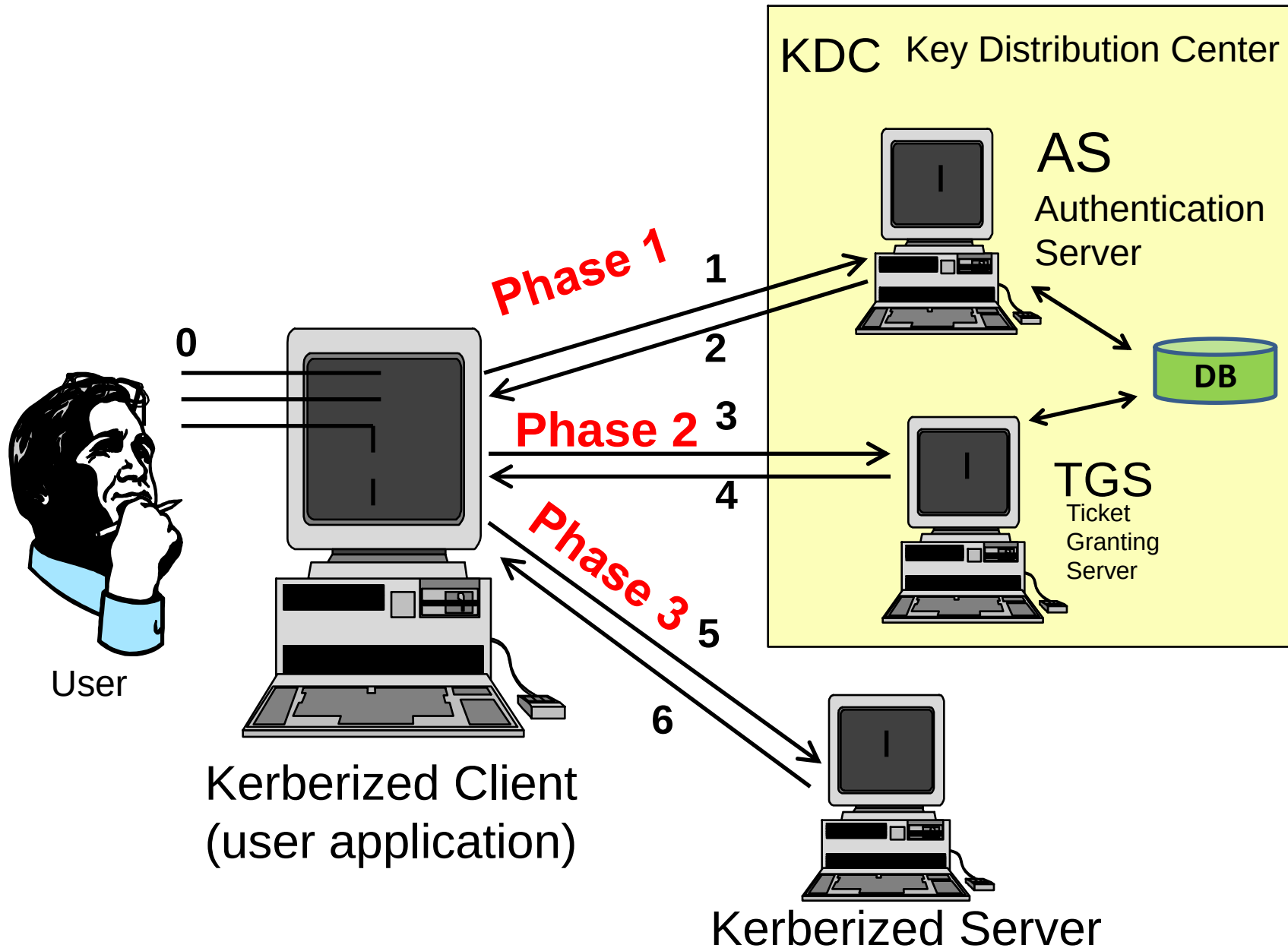
- Kerberos is for authentication only and provides Single Sign-on (SSO)
- LDAP can be used for authentication, authorization, and name services (no SSO)
- Active Directory is a kerberized directory service with an LDAP interface
- **Use Kerberos for authentication, LDAP for authorization and name services**

Kerberos Overview - Realms

- A Kerberos environment consists of:
 - **A Kerberos server** (KDC)
 - A number of **clients**, all registered with server and sharing keys with server.
 - **Application servers**, sharing keys with server
- This is termed a realm
 - Typically a single administrative domain

- ~~• If multiple realms, their Kerberos servers must share keys and trust~~

Kerberos Overview

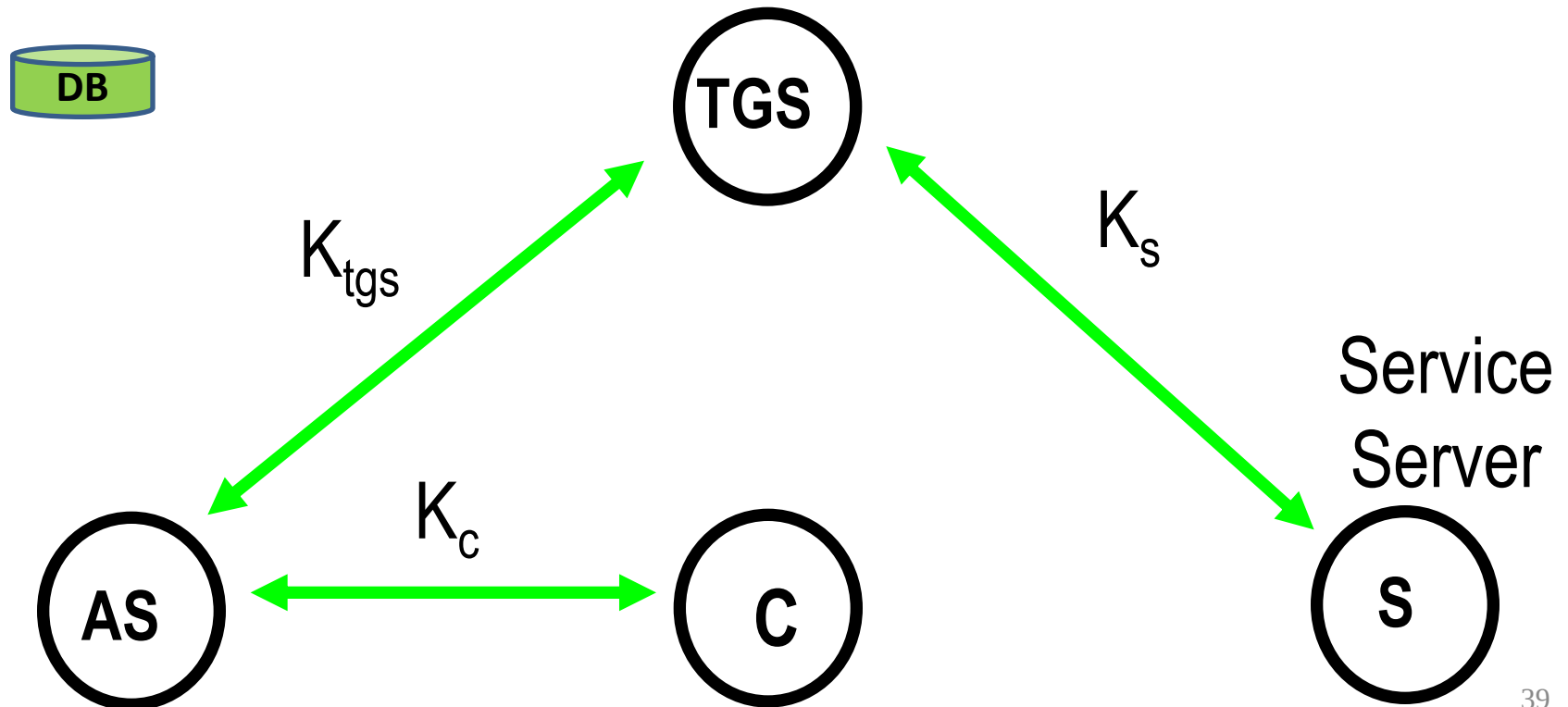


Kerberos Overview

- A basic third-party authentication scheme
- The Key Distribution Center (KDC) have two parts:
- Have an Authentication Server (AS)
 - (Phase 1) Users application initially negotiate with AS to identify the user himself
 - (Phase 2) AS provides a non-corruptible authentication credential (ticket granting ticket TGT)
- Have a Ticket Granting server (TGS)
 - (Phase 3) Users subsequently request access to other services from TGS on basis of users TGT

Static Kerberos Shared Secrets

- Chiavi private condivise, contenute (anche) nel DB
- Definite quando si configura il KDC (caso Ktgs)
 - ... si inserisce un nuovo utente (Kc)
 - ... si inserisce un nuovo servizio (Ks)

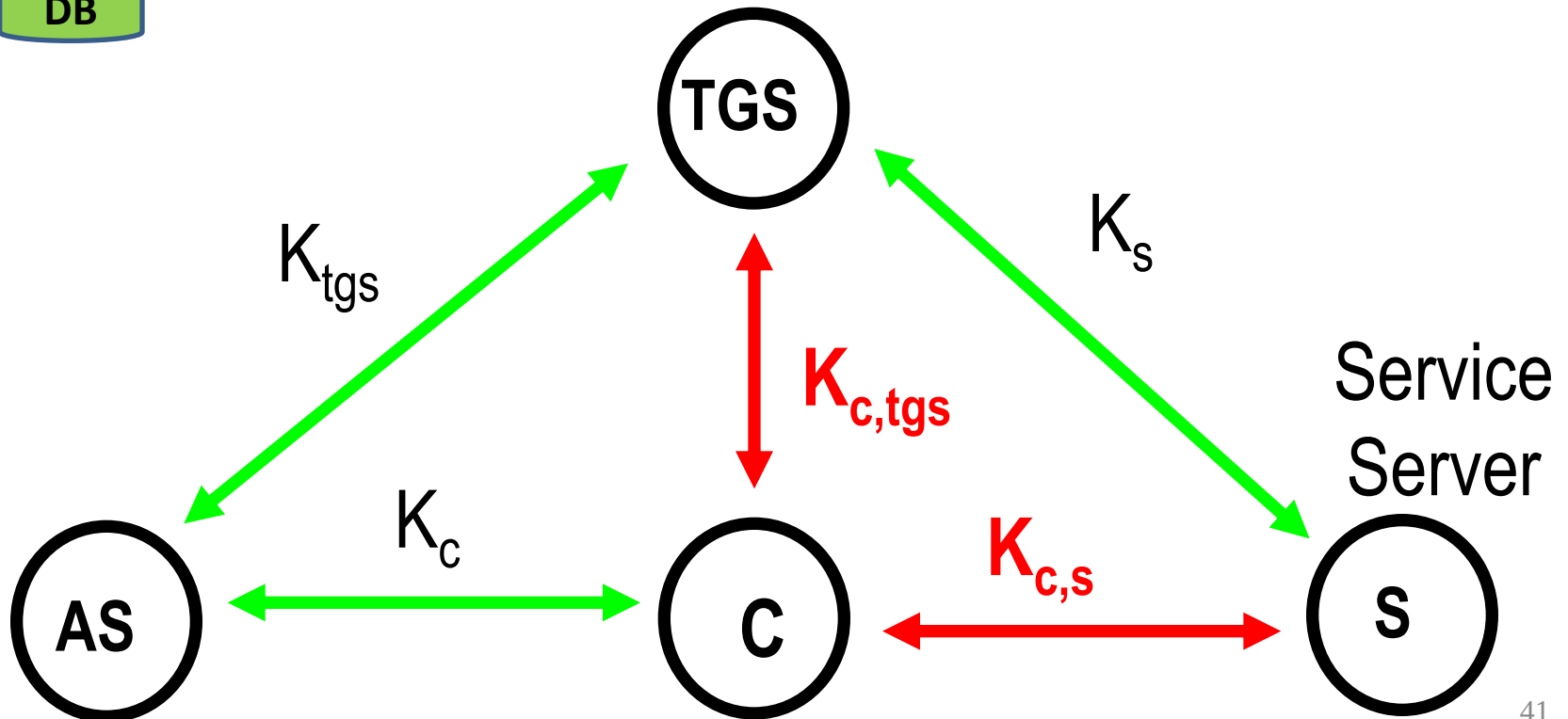


Creazione Chiavi Statiche di cifratura

- La chiave K_c ,
 - che un utente C usa per criptare le comunicazioni con l'Authentication Server,
 - è ricavata
 - dalla password dell'utente per il realm dell'AS,
 - applicandovi una funzione Hash.
- La funzione di libreria `string2key` trasforma la password in una chiave di cifratura simmetrica.
- Tale trasformazione è diversa in funzione di quale algoritmo di crittografia si intende utilizzare (DES, Triplo DES).

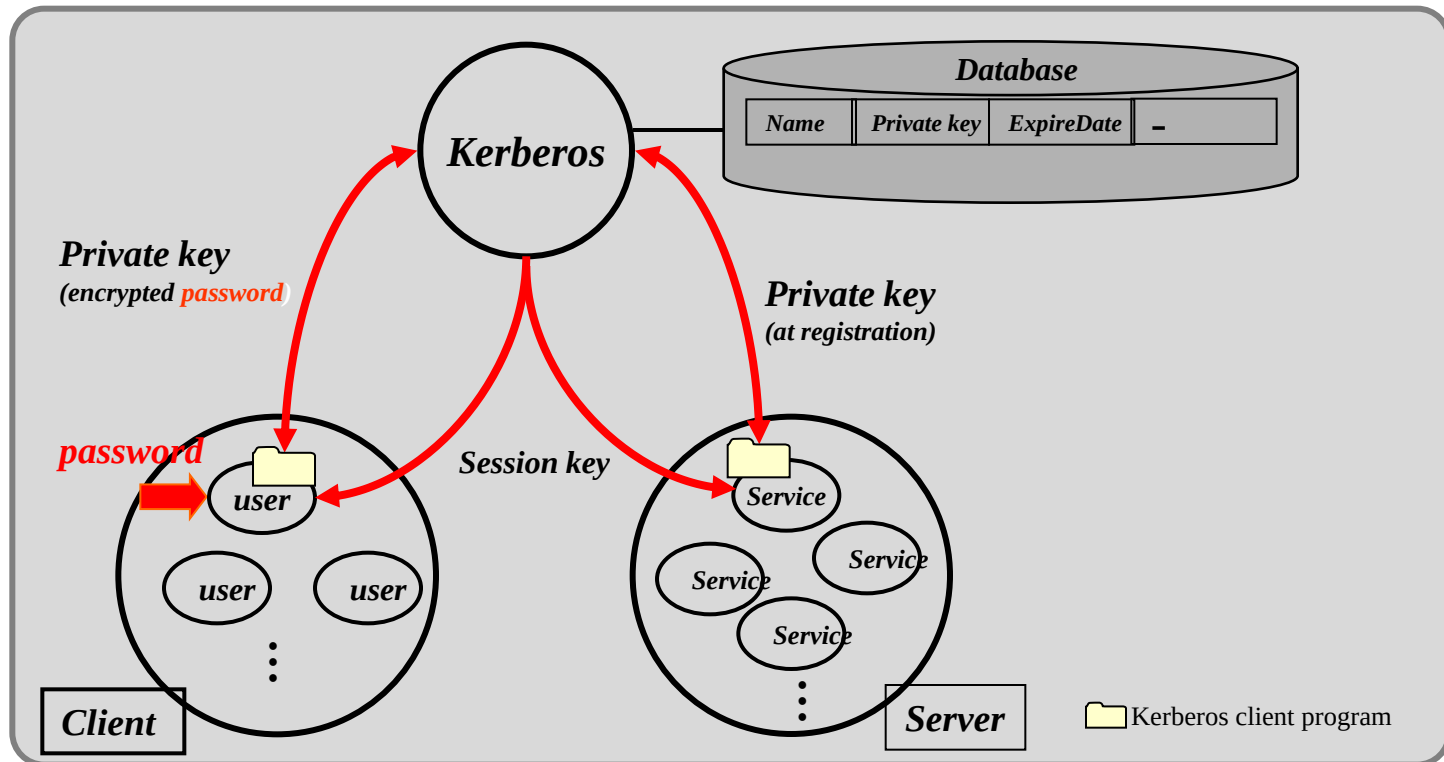
Run-time Kerberos Shared Secrets

- **Chiavi di Sessione** - Chiavi private condivise, create a run-time su richiesta dell'utente C che richiede il servizio V
 - **$K_{c,tgs}$**
 - **$K_{c,s}$**



Kerberos data flows

- Trusted third-party authentication service
- Based on Needham and Schroeder key distribution algorithm
- Ticket = EKs [server ; client ; address ; timestamp ; lifetime ; Ks,c]



Kerberos v5 Protocol Overview

Come già detto il protocollo di autenticazione Kerberos si basa ed estende il modello di distribuzione delle chiavi di Needham e Schroeder

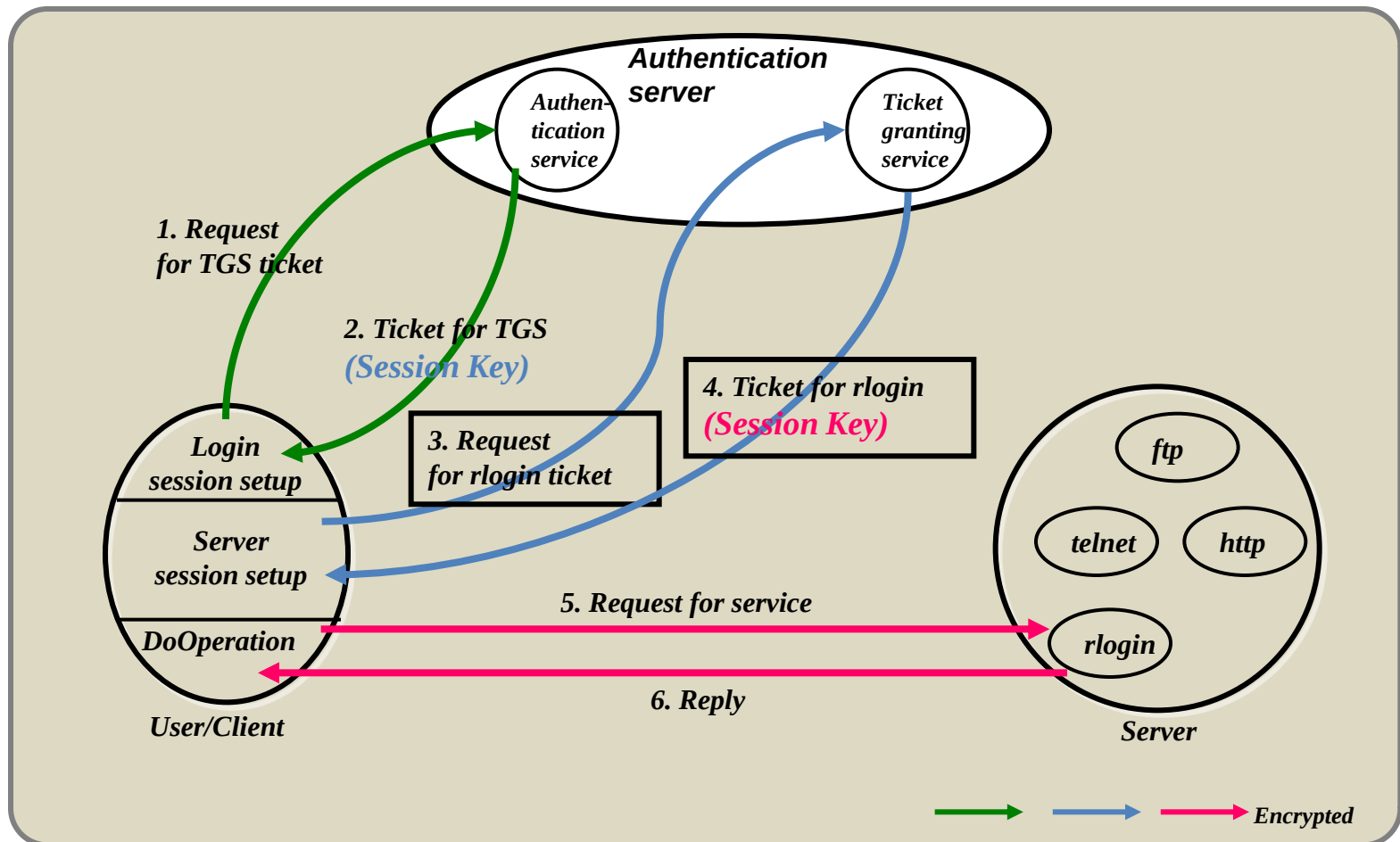
In generale si divide in tre fasi:

1. L'utente richiede ed ottiene "le credenziali" dal server di autenticazione, per accedere al servizio Tgs.
2. L'utente richiede ed ottiene le credenziali dal Tgs, per un servizio specifico.
3. L'utente presenta le credenziali ottenute dal Tgs, al server del servizio richiesto.

Queste credenziali altro non sono che i ticket e gli authenticator.

Kerberos v5 Protocol Overview

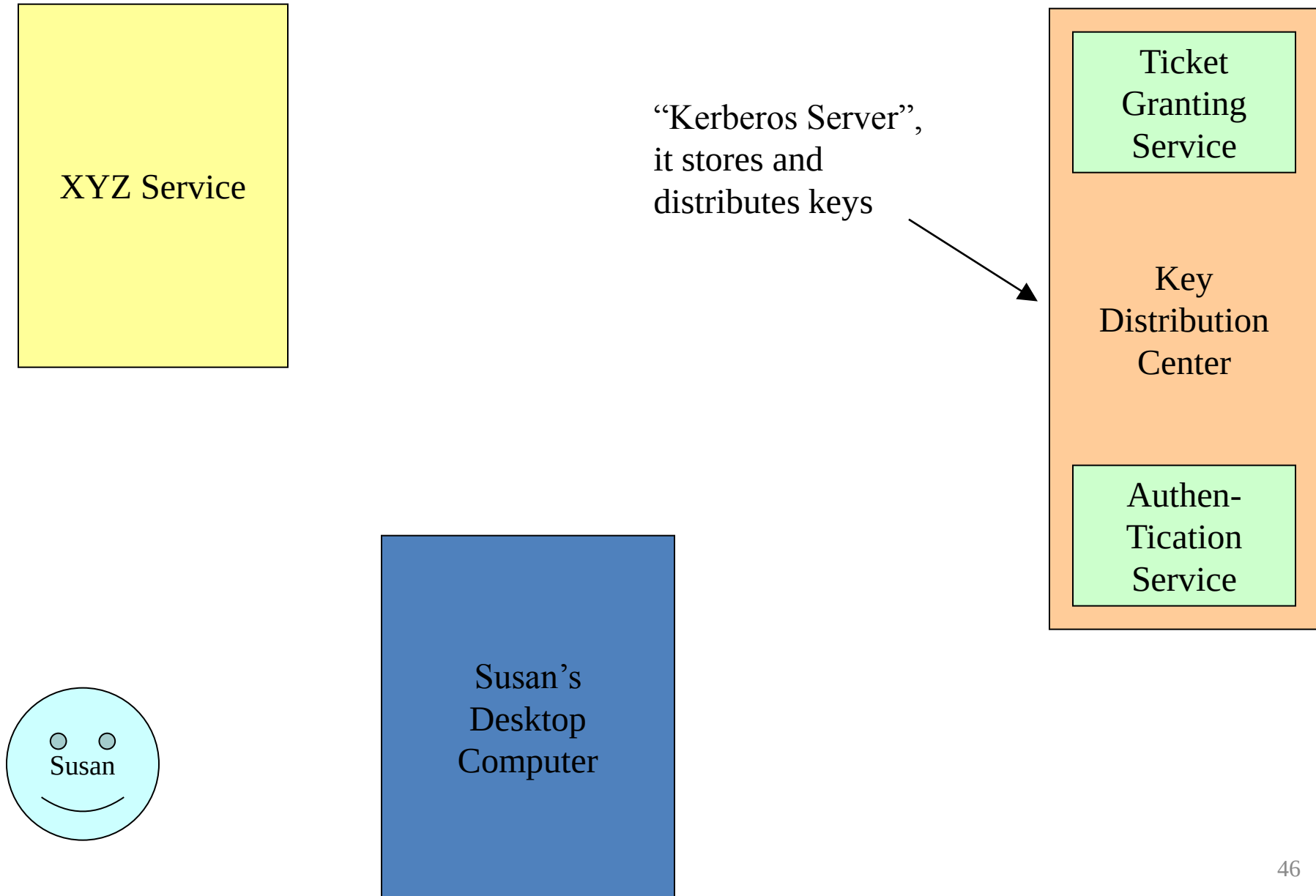
Phases



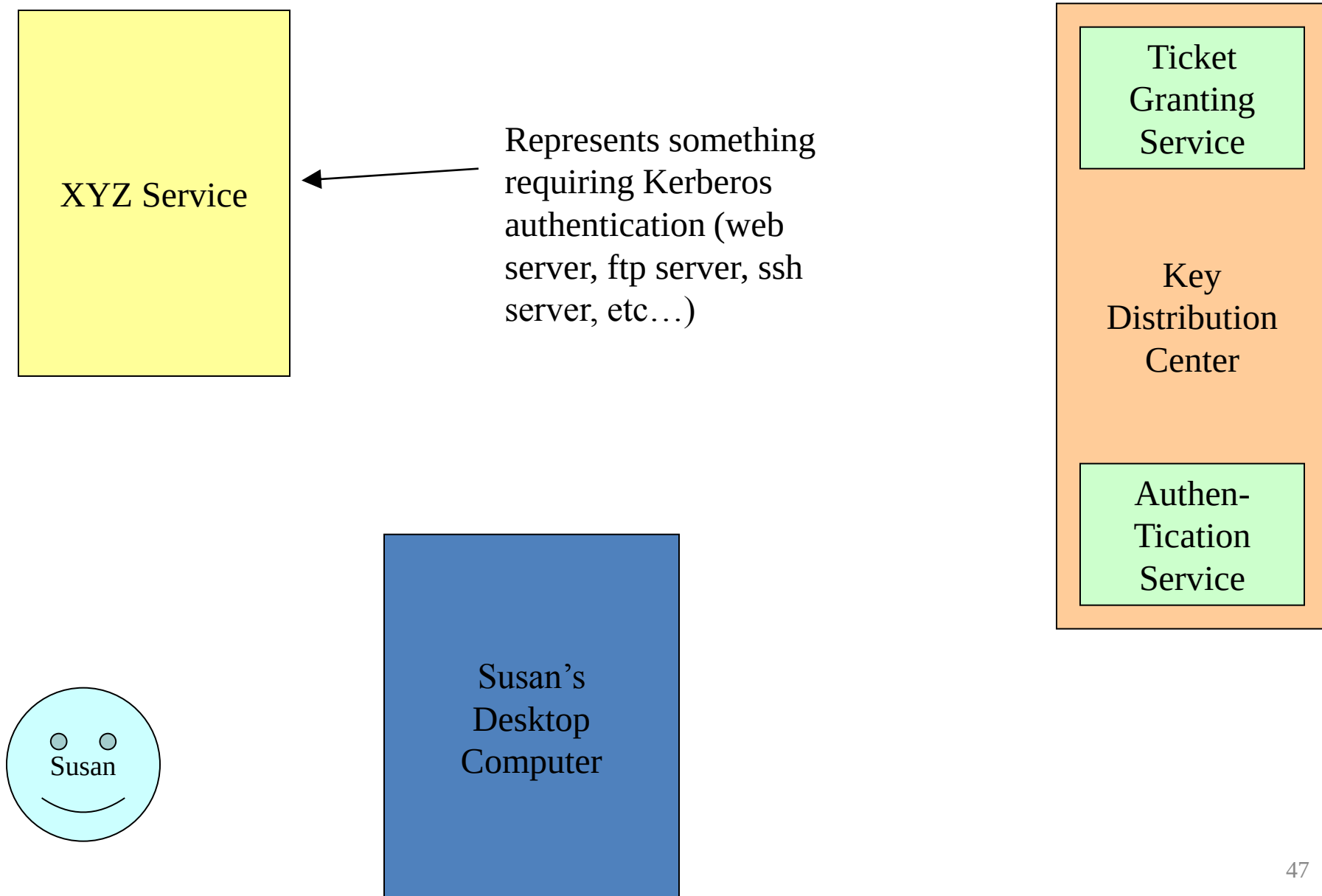
SPIEGAZIONE DEL MECCANISMO DI CREAZIONE e PASSAGGIO CHIAVI

- Le prossime slide illustrano il meccanismo di passaggio delle chiavi di sessione

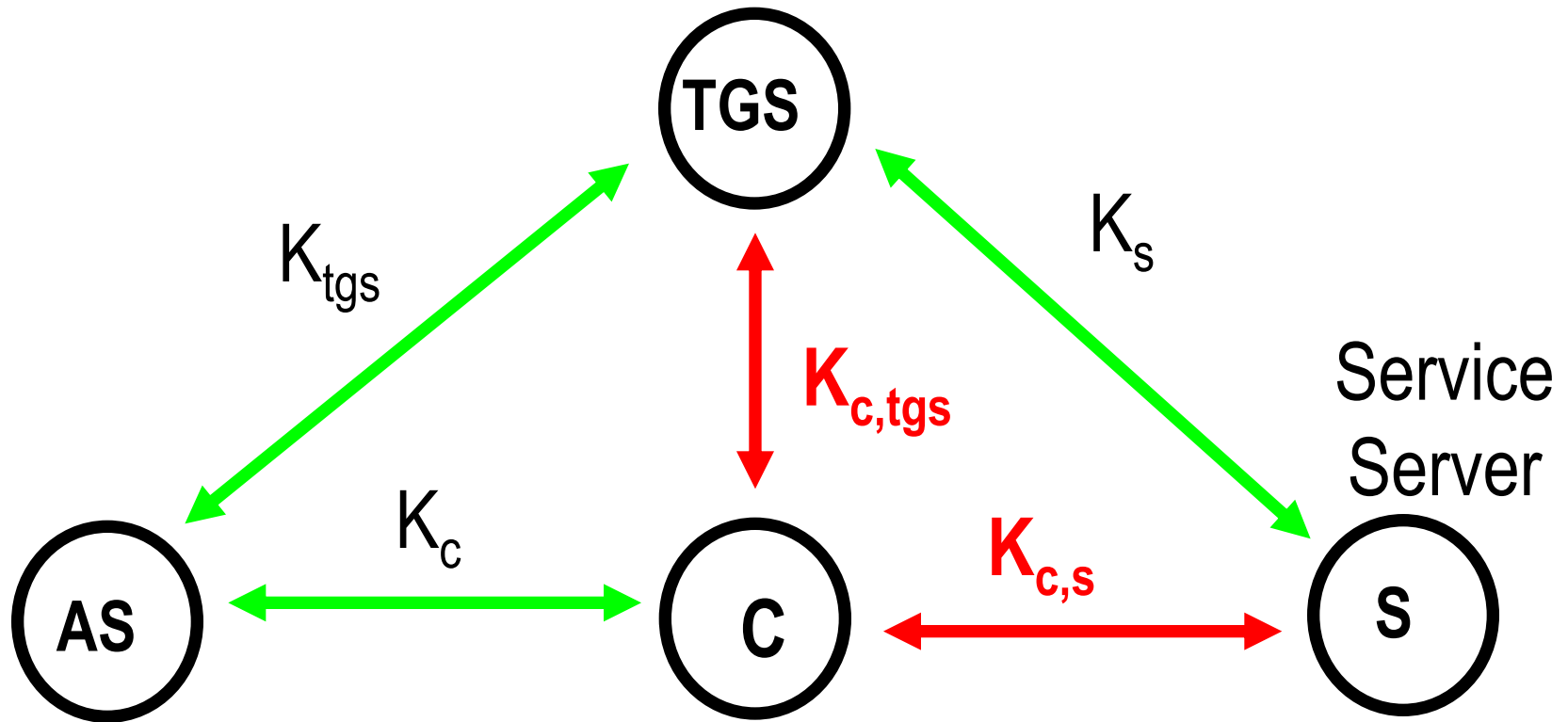
ATTORI - Centro Distribuzione Chiavi



ATTORI - Fornitore di Servizi



Kerberos - Attori di un dominio Realm



Kerberos - Notazioni

Messaggio o parte di messaggio = sequenza di campi separati da ;
Options ; IDc ; Realmc ; IDtgs ; **EKc** [Times] ; Nonce1

Chiave = $K_{\text{elenco_con_virgole}}$
Ktgs **Ktgs** **Kc,tgs** **Kc,s**

Messaggio o parte di messaggio **CRIPATATO CON CHIAVE** SIMMETRICA

E*Chiave* [messaggio]
EKc [IDtgs ; Times2]

Identificatore di una entità (univoco solo all'interno di un dominio)
IDc IDs

Identificatore di un dominio Realm a cui appartiene l'entità c
Realmc

Kerberos FASE 1

Protocollo Kerberos Version 5 Message Exchange:1

- To obtain ticket-granting ticket:

(1) **C** → **AS** :

(Options ; IDc ; Realmc ; IDtgs ; **EKc [Times1]** ; Nonce1)

where

Times1 = Time1 ; Lifetime1

(2) **AS** → **C** :

(Realmc ; IDc ; **Ticket tgs** ;

EKc [Kc,tgs ; IDtgs ; Times2 ; Nonce1 ; Realm tgs])

where

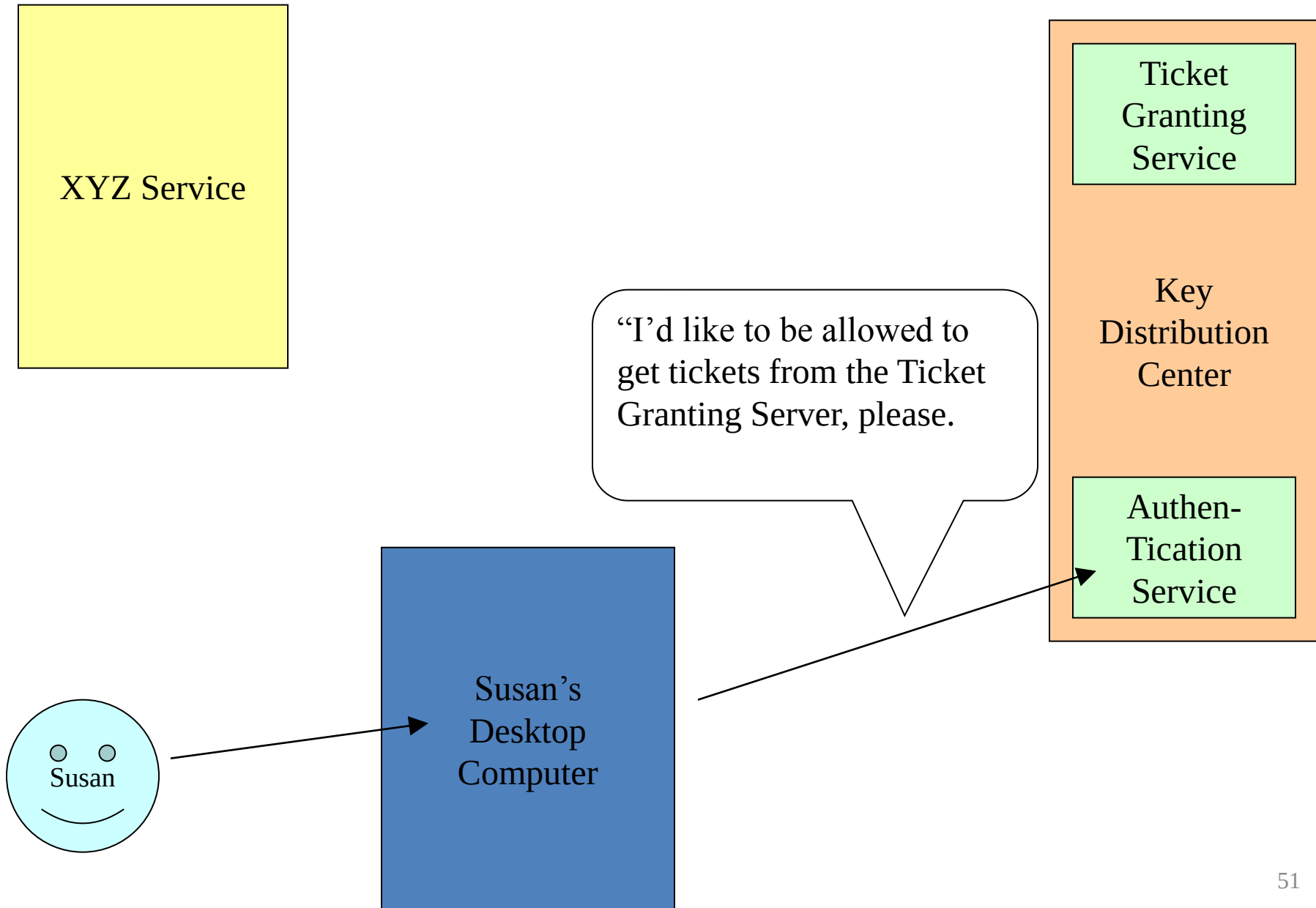
Ticket tgs = **EKtgs [Flags ; Kc,tgs ; Realm c ; IDc ; ADc ; Times2]**

Times2 = Time2 ; Lifetime2

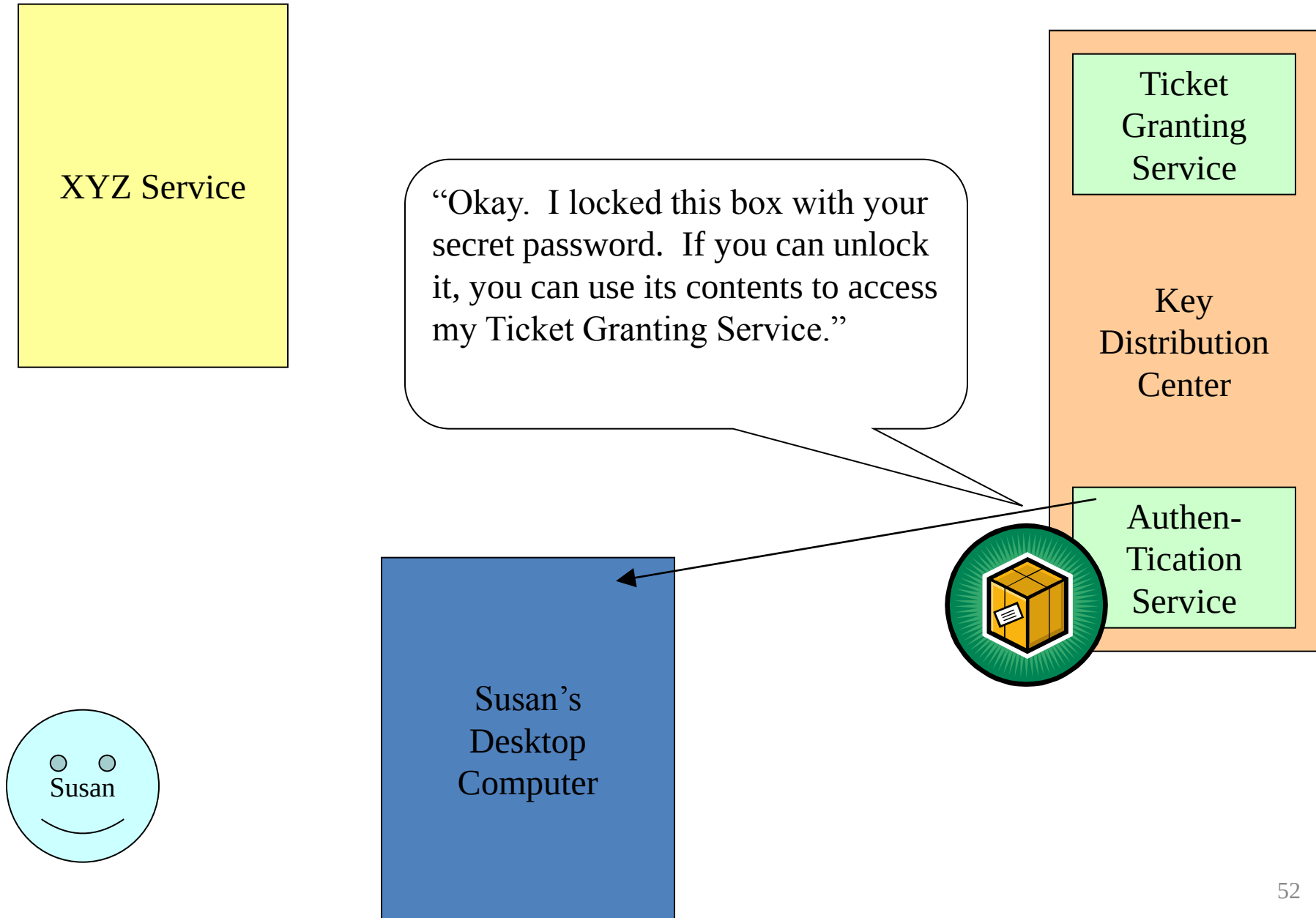
ADc = network address machine C

pre-authentication

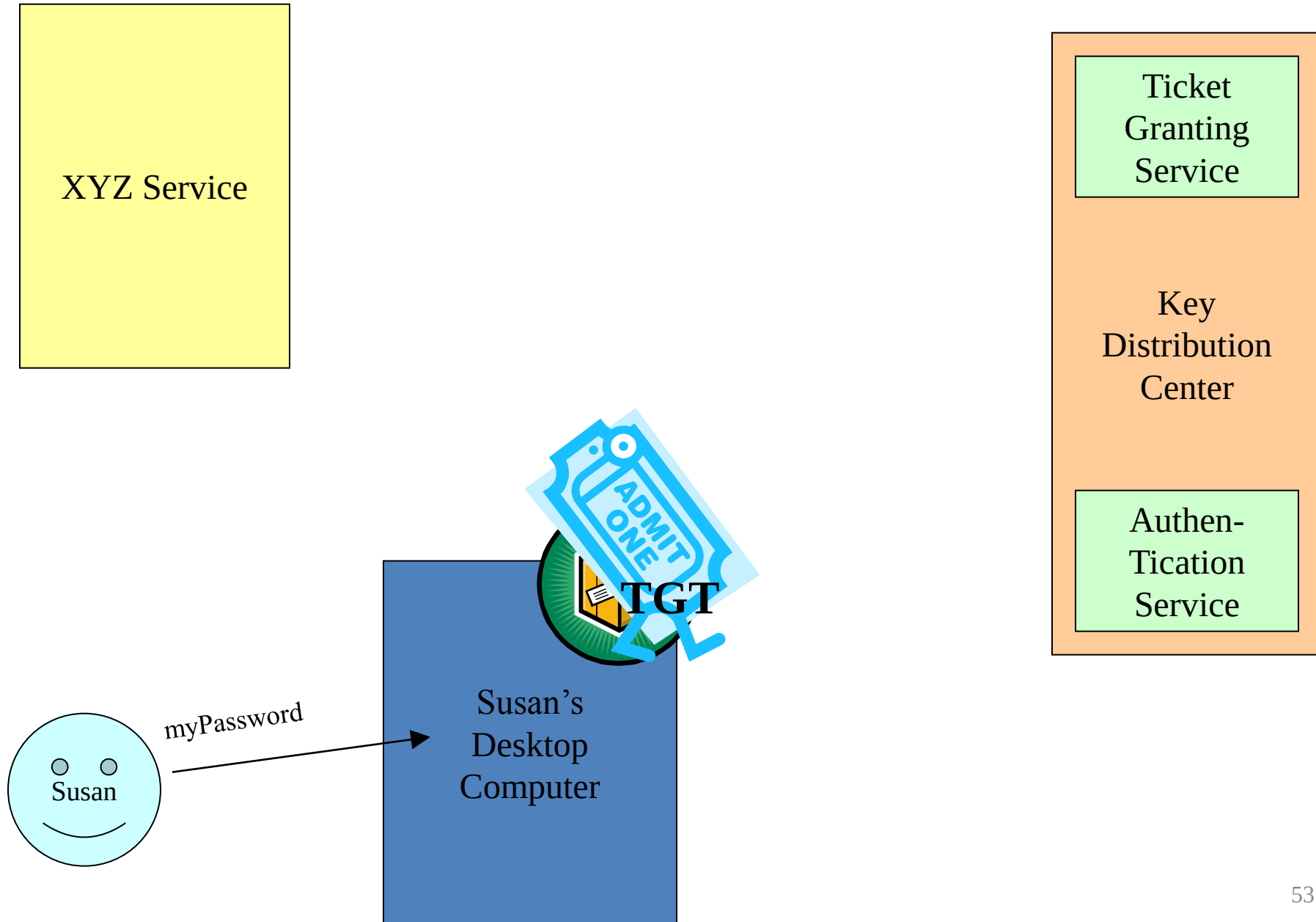
FASE 1 - richiesta



FASE 1 - risposta



FASE 1 - decodifica risposta



FASE 1 - risultato abilitato a svolgere Fase 2



Because Susan was able to open the box (decrypt a message) from the Authentication Service, she is now the owner of a shiny “Ticket-Granting Ticket”.

The Ticket-Granting Ticket (TGT) must be presented to the Ticket Granting Service in order to acquire “service tickets” for use with services requiring Kerberos authentication.

The TGT contains no password information.

Kerberos FASE 2

Protocollo Kerberos Version 5 Message Exchange:2

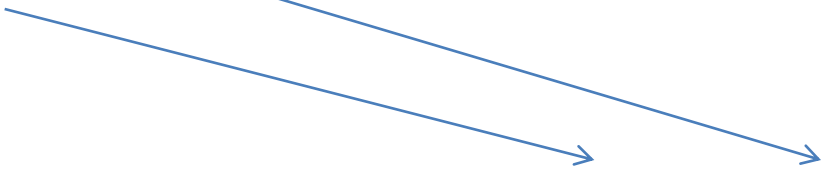
- To obtain service-granting ticket :

(3)C → TGS :

Options ; IDs ; Times2 ; Nonce2 ; **Ticket tgs** ; **Authenticator c**

(4)TGS → C :

Realmc ; IDc;**Ticket s**; **EK c,tgs** [Kc,s ;Times2; Nonce2 ; IDs ; Realm s]



where

Ticket tgs= **EKtgs** [Flags ; Kc,tgs ; Realm c ; IDc ; ADc ; Times2]

Times2 = Time2 ; Lifetime2

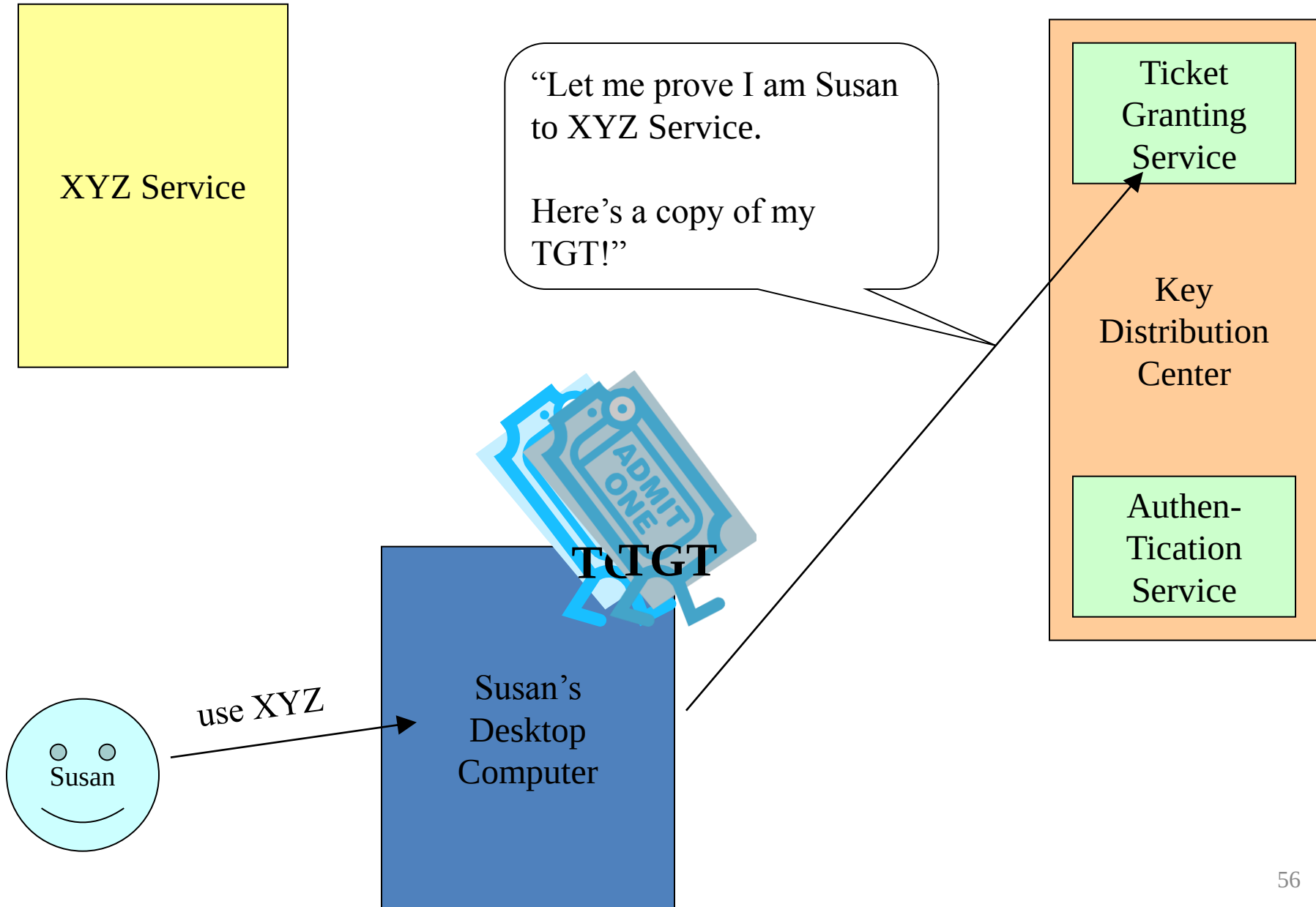
Ticket s : **EK s** [Kc,s ; Realmc ; IDc ; ADc ; Times3]

Times3 = Time3 ; Lifetime3

Authenticator c : **EK c,tgs** [IDc ; Realmc ; **Time3**]

ADc = network address machine C

FASE 2 - richiesta



Why Authenticator ?

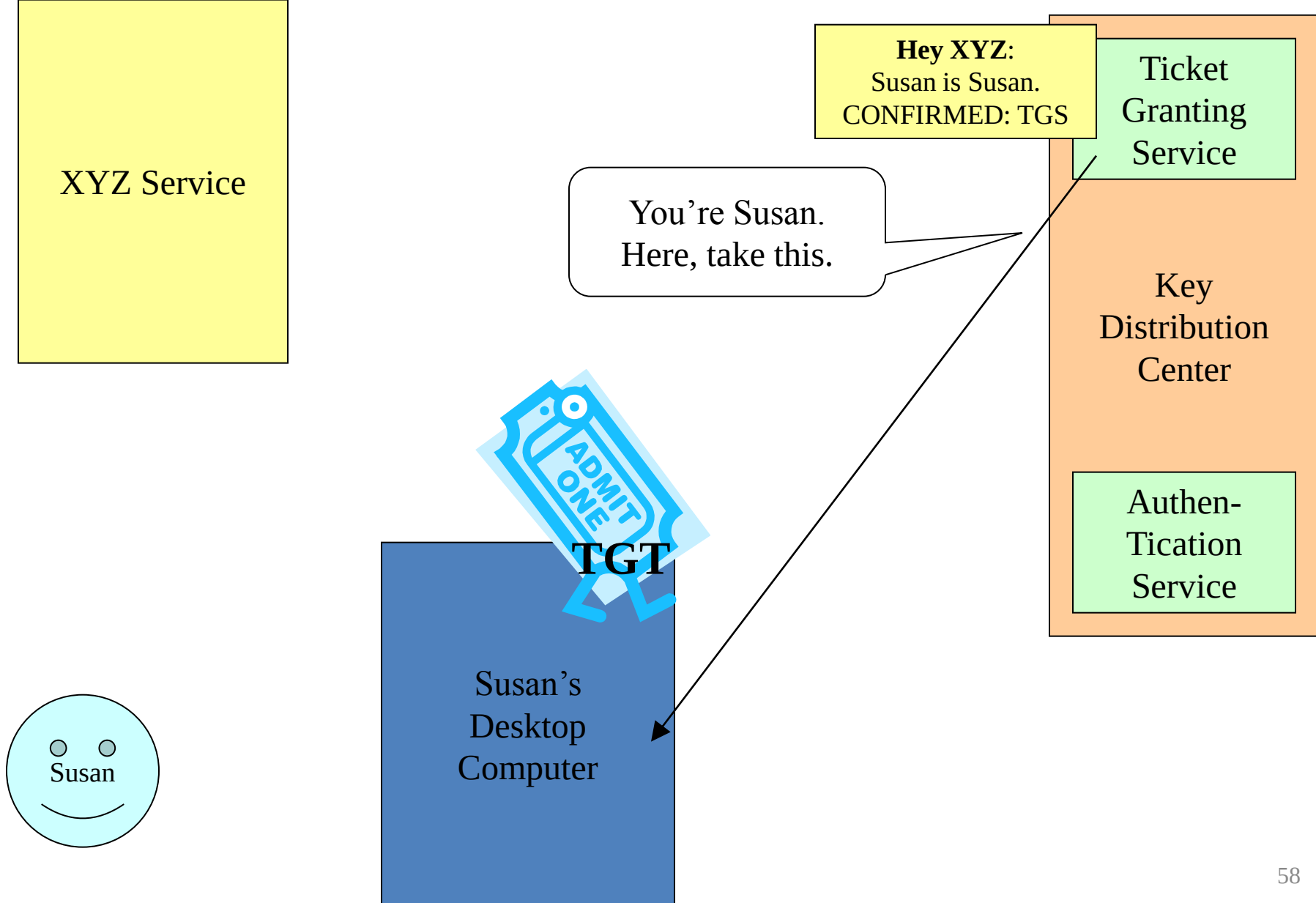
$$\text{Authenticator}_C = \text{EK}_{C,\text{tgs}}[\text{ID}_C ; \text{RealmC} ; \text{Time}_3]$$

Notes: Authenticator created by user to assure TGS that ticket presenter is same as user for whom ticket was issued; intended for one time use; timestamp limits replay

$K_{C,\text{tgs}}$ – session key available to user; permits secure exchange for TGS & ID_C for the session

Time_3 – time stamp created for this authenticator

FASE 2 - risposta



Kerberos FASE 3

Protocollo Kerberos Version 5 Message Exchange:3

- To obtain service :

(5) $C \rightarrow S :$

Options ; Ticket s ; Authenticator c

(6) $S \rightarrow C :$

EK c,s [Time4 ; Subkey ; Seq#]

where

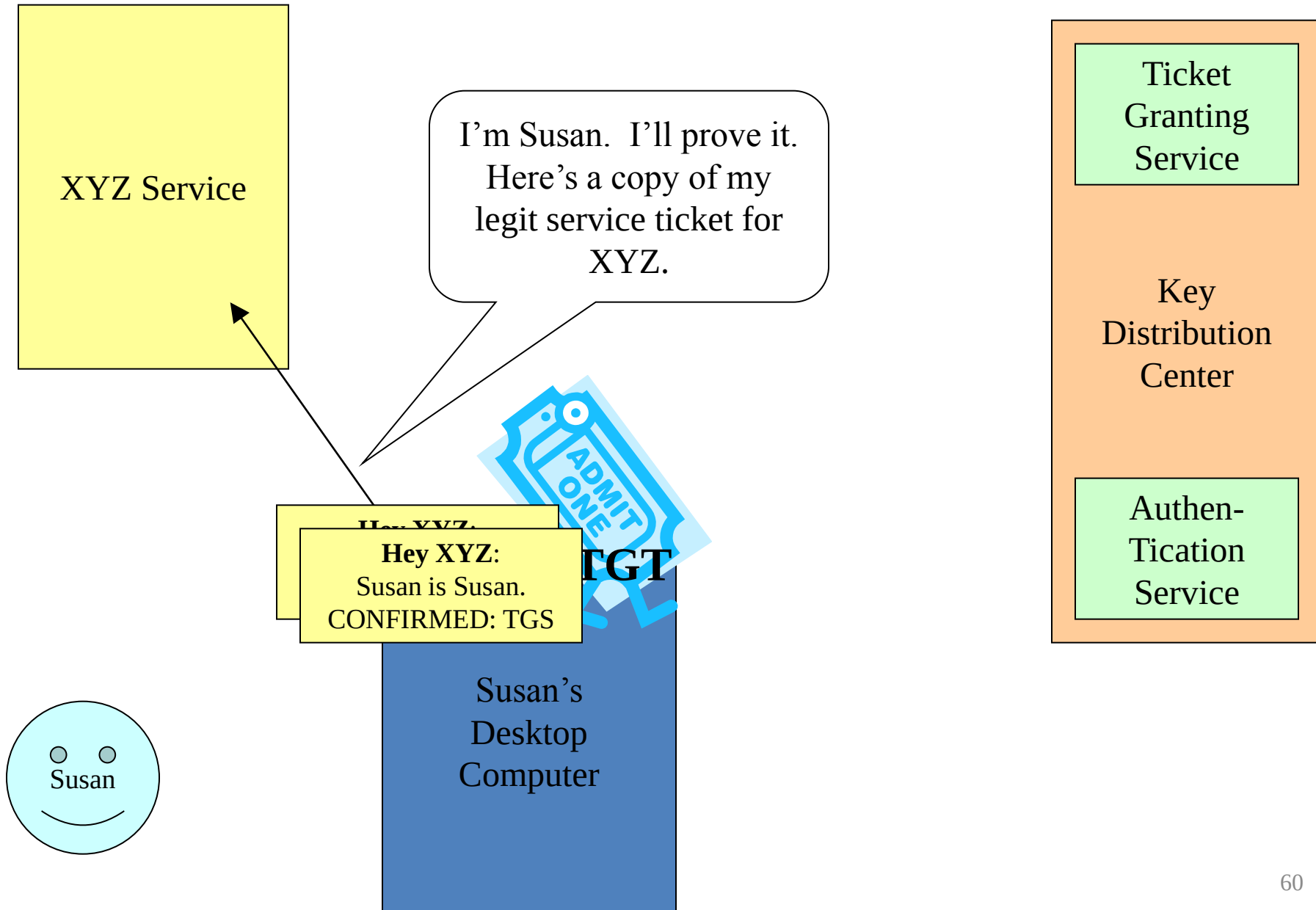
Ticket s = **EK s** [Flags ; $K_{c,s}$; Realm c ; ID c ; AD c ; Times3]

Authenticator c = **EK c,s** [ID c ; Realm c ; Time4 ; Subkey ; Seq#]

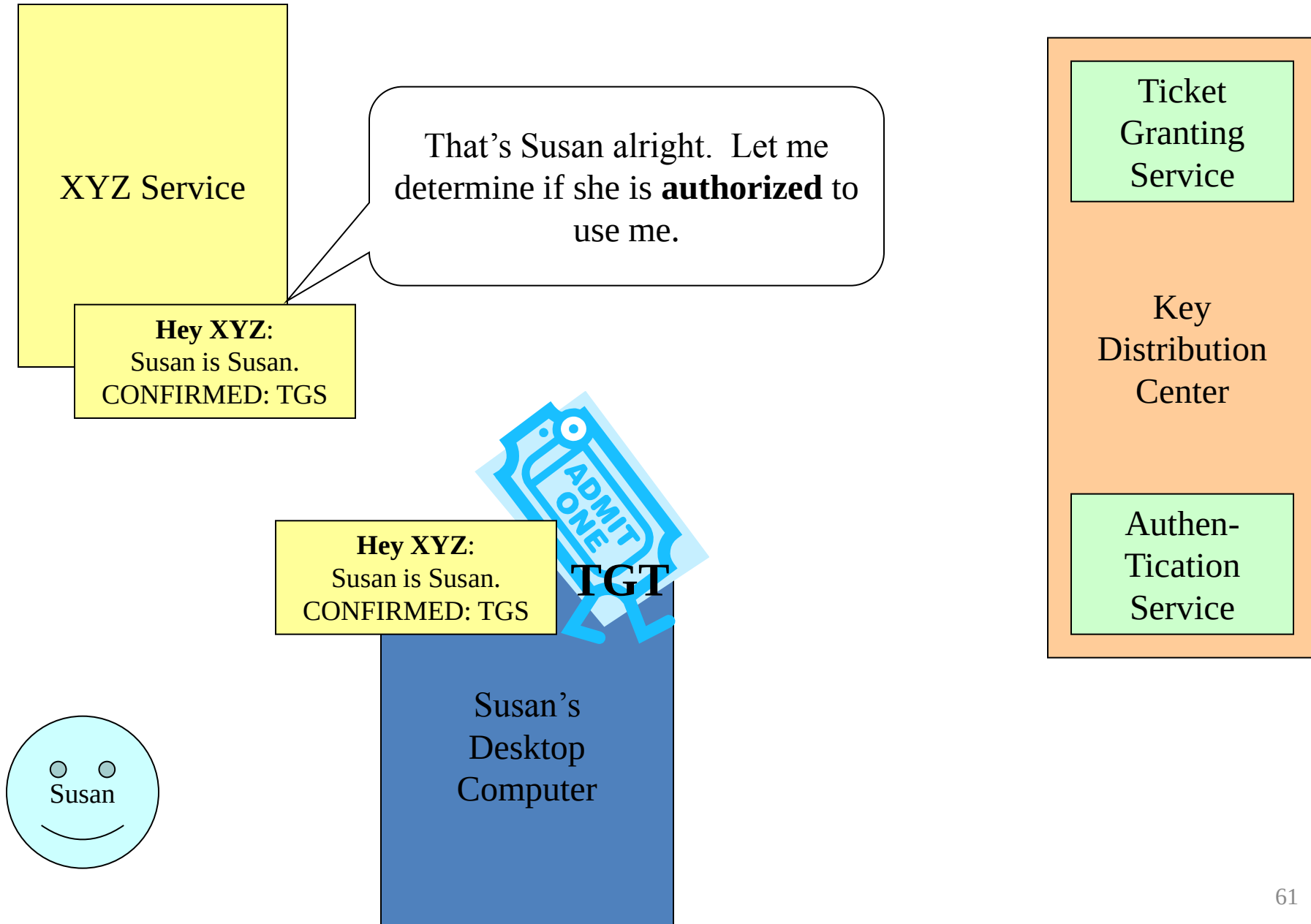
Times3 = Time3 ; Lifetime3

AD c = network address machine C

FASE 3 - richiesta servizio



FASE 3 - verifica autenticazione e ...

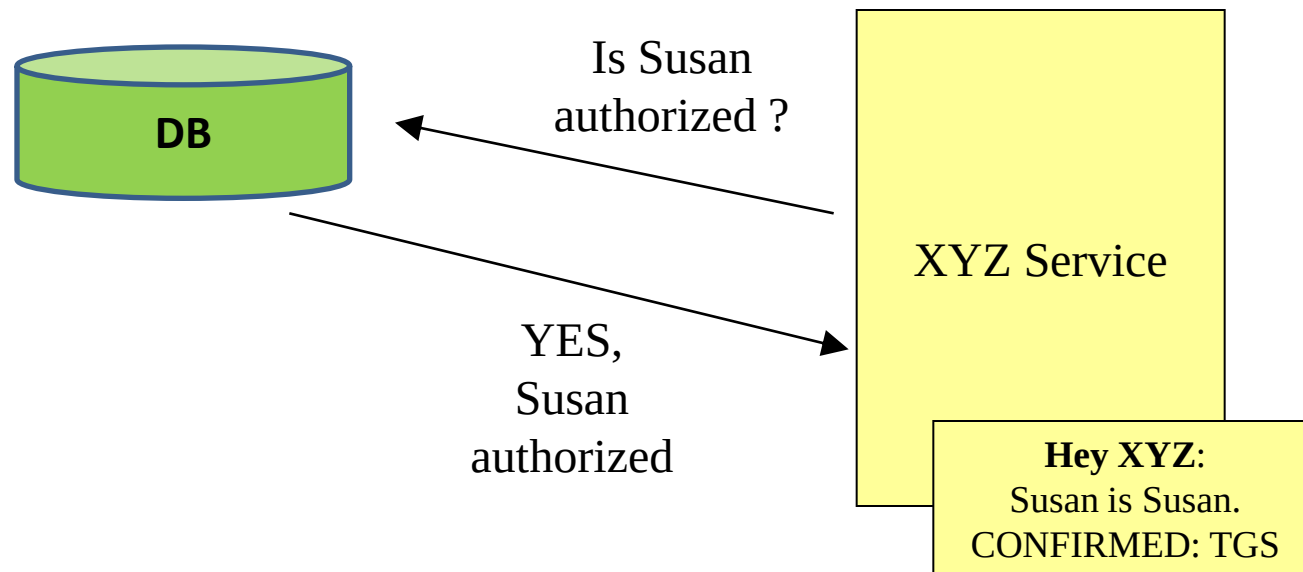


DOPO FASE 3 - verifica autorizzazione

Non fa parte del meccanismo di distribuzione chiavi

Authorization checks are performed by the XYZ service...

Just because Susan has **authenticated** herself **does not** inherently **mean** she is **authorized** to make use of the XYZ service.



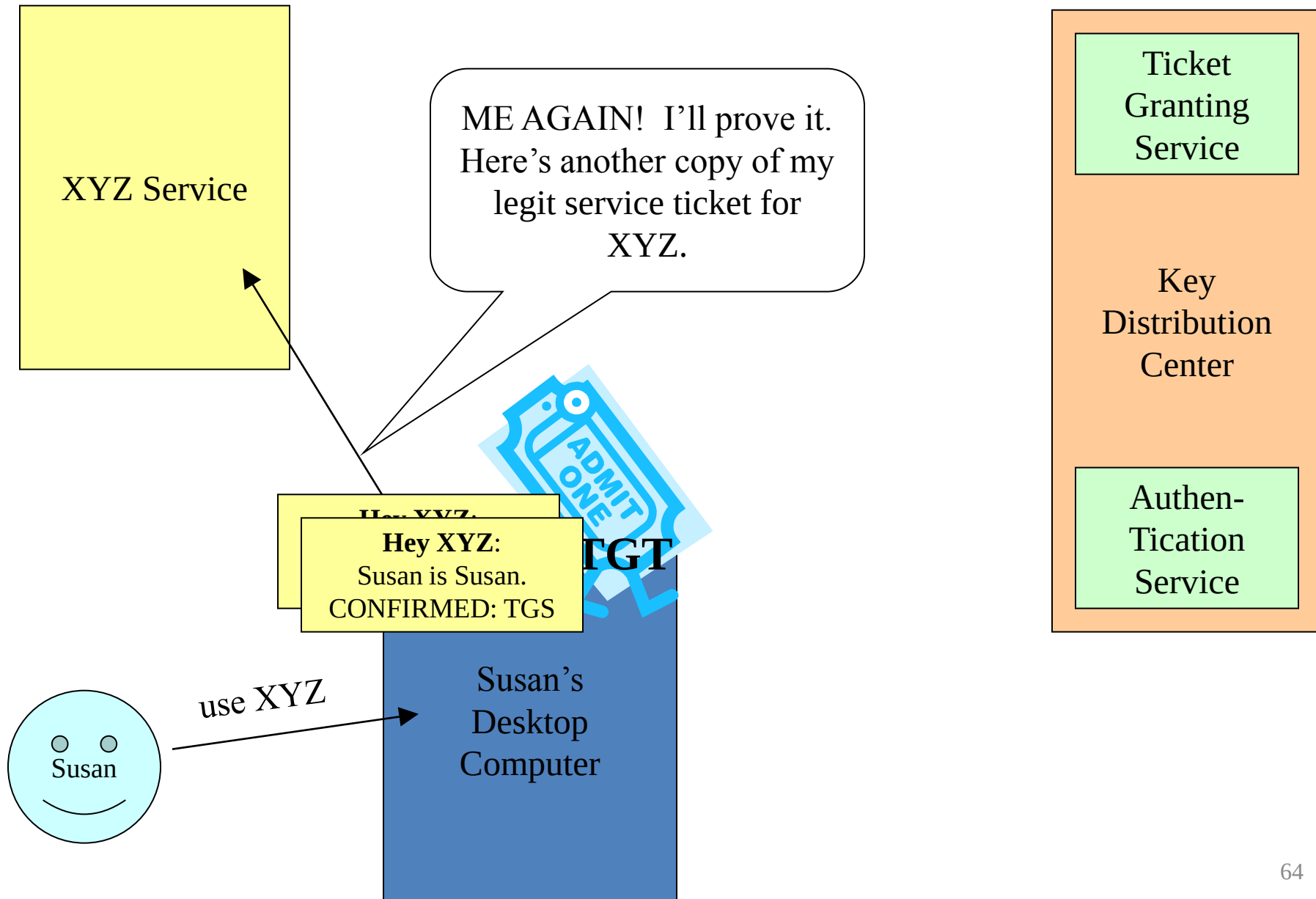
FASE 3 - verifica autenticazione

One remaining note:

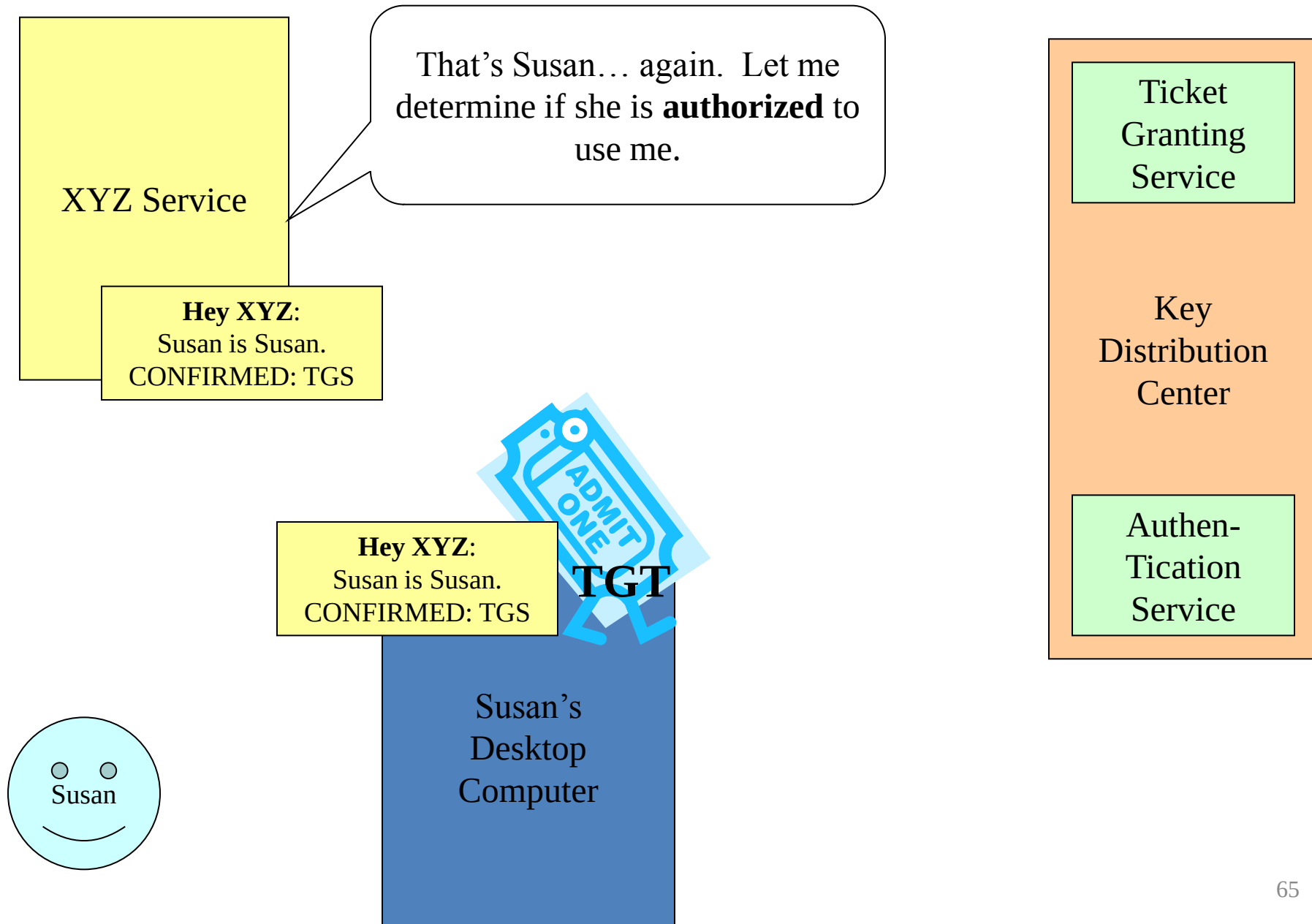
Tickets (your TGT as well as service-specific tickets) **have expiration dates** configured by your local system administrator(s). **An expired ticket is unusable.**

Until a ticket's expiration, it may be used repeatedly.

FASE 3 bis, tris, ... - altra richiesta servizio



FASE 3 bis, tris - verifica autenticazione



Una piccola aggiunta sui tempi

- Il campo Times che ho definito come formato da due campi,
- in realtà può prevedere un terzo campo poiché in kerberos è possibile chiedere di rinnovare i ticket
- Times
 - From: the desired start time for the ticket
 - Till: the requested expiration time
 - Rtime: requested renew-till time

Kerberos : Strengths

- User's passwords are never sent across the network, encrypted or in plain text
- Secret keys are *only* passed across the network in encrypted form
- Client and server systems mutually authenticate
- It limits the duration of their users' authentication.
- Authentications are **reusable** and **durable**
- Kerberos has been scrutinized by many of the top programmers, cryptologists and security experts in the industry

KERBEROS – il DataBase

Il Database - cuore di Kerberos

- parte fondamentale del sistema kerberos
- conserva all'interno tutte le chiavi e gli identificatori
- risiede sul server di autenticazione AS
- è soggetto sia a lettura che a scrittura
- ad ogni accesso Kerberos aggiunge/toglie record dai vari campi che compongono il database



Struttura

Le voci del database

- **NAME**: contiene indicatori dei Principal
- **KEY**: contiene le chiavi crittografiche dei Principal
- **PKVNO** (Principal Key Version Number): versione della chiave dei Principal
- **MAX LIFE**: contiene durata massima vita del ticket-Principal
- **MAX RENEWABLE LIFE**: contiene durata massima vita del ticket rinnovabili

4. Cifratura & Sicurezza

Algoritmi usati

- la parte cifrata riveste un ruolo fondamentale all'interno dei singoli passaggi legati al processo di scambio delle credenziali alla base del Kerberos
- **string2key**: funzione che converte password in chiave di cifratura simmetrica
 - come? → applicare una funzione HASH alla stringa composta concatenando la password dell'utente con un "SALT" il cui valore varia a secondo delle versioni di Kerberos e che serve a rendere la trasformazione sulla password dipendente dalla macchina sulla quale è effettuata.
- per implementare lo scambio di messaggi tra AS, client e server, sono necessari tre elementi:
 - una algoritmo di crittografia forte;
 - una funzione HASH;
 - una funzione che implementi il checksum per l'Authenticator;
- nelle prime versioni di Kerberos l'alg. di crittografia usato era solo il DES, e non si usava il SALT
- nella versione 5 (attuale) algoritmi disponibili sono
 - **DES** e **Triplo DES** per la crittografia (si ottiene chiave da 64 bit)
 - **DES-CBC** per l'hash
 - **CRC32** per i checksum

5 Bugs & Attacks

Punti deboli di Kerberos

1. Nella gestione dei ticket è possibile per un pirata intromettersi facendo credere a Kerberos di essere un client,
(**SPOOFING ATTACK**) avendo il totale controllo della rete.
 - Ma come avviene?
 1. pirata tramite Backdoor, trojan prende possesso di un pc
 2. lancia uno sniffer invisibile
 3. intercetta un ticket, preleva le credenziali e si spaccia per il client al quale il ticket era riferito
2. Nella risposta di Kerberos ad un principal, possibilità di intercettazione della chiave privata rilasciata (attacco **BRUTE FORCE**)
3. Nella versione 5 trovate due falle con possibili attacchi **Ddos** che consiste nel creare carico di lavoro elevato tale da bloccare il sistema
 - 1a falla: tipo "**double-free**" e potrebbe essere sfruttata da un cracker per compromettere l'intera infrastruttura di autenticazione di una rete.
 - 2a falla: tipo "**buffer overflow**", quindi potrebbe essere utilizzata da un aggressore sia per mandare in crash un server per la distribuzione delle chiavi Kerberos sia per eseguire del codice

5. Bugs & Attacks (2)

Altri attacchi

- **Attacco alle credenziali di amministratore Kerberos**

un pirata scopre la password dell'amministratore principale con la quale si ha completo accesso al database di kerberos

- **Denial of service**

creazione carico elevato

- **The Insider**

Kerberos non è protetto da possibili modifiche dannose ai privilegi da parte di utenti interni (come amministratori)

- **Social engineering and password exposure**

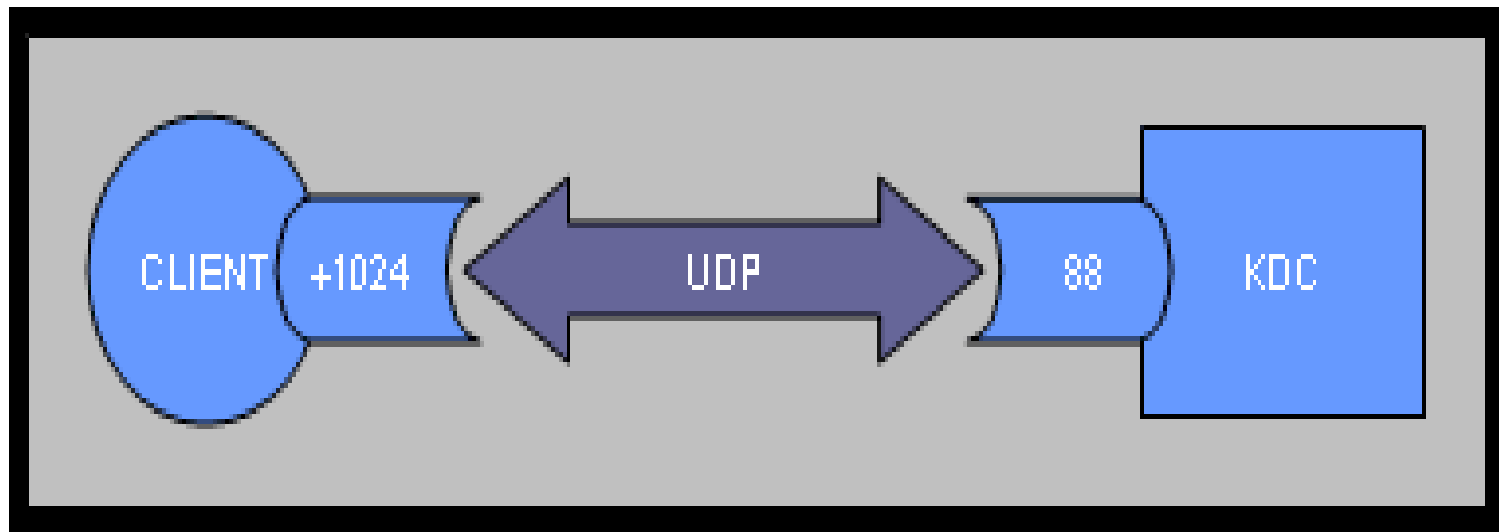
Anche in questo caso, Kerberos non è protetto da possibili divulgazioni non autorizzate da parte degli utenti, di password, impostazioni di sicurezza

- **Security holes in the Kerberos software itself**

Questo è un rischio costante, perché nonostante gli sviluppi nella progettazione di software, è davvero difficile creare software sicuro e stabile.

KERBEROS & Trasporto

Il sistema Kerberos, su reti TCP/IP, utilizza come trasporto il protocollo non connesso UDP e come porta per il servizio, la porta **88** per **KDC** e **1024** per il **Client**.



Kerberos Limitations

- **Single Failure Problem:** If the AS is down, no user can access any resources. So Kerberos is prone to **denial-of-service (DoS)** attacks. **Duplicated AS?** Possible, but not easy to maintain.
- **Clock Synchronization** is needed, since timestamps are used. Reasonable time interval for clock skew?
 - **Too short:** Rejecting many valid requests.
 - **Too long:** Suffering replay attack.
- **Limited Scalability:** Usually, the AS can support with hundreds of thousands users. Suitable for a university but not for the Internet, where PKIs with digital certificates are better.
- **Off-line Password Attacks:** Kerberos is vulnerable to this kind of attacks since a message is encrypted with a key derived from the client's password. **The authenticator limits this issue** because limits the number of AS responses.

Sviluppare applicazioni che usano Kerberos

Dobbiamo distinguere due classi di applicazioni che usano Kerberos:

- Le applicazioni che svolgono il ruolo di Service Provider (server).
- Le Applicazioni (client) direttamente usate dall'utente per accedere ai servizi forniti dal Service Provider.

Le API di Kerberos v5 sono piuttosto complicate da utilizzare.

Esiste uno standard rilasciato da IETF che definisce delle API che permettono di utilizzare dei servizi di sicurezza tra i quali i servizi di Kerberos. Questo standard è chiamato **Generic Security Service Application Program Interface** (GSSAPI o GSS-API), definito nel 2000.

Purtroppo, anche GSS-API è un po' complicato da usare, perciò è stato definito un'ulteriore standard **Simple Authentication and Security Layer (SASL)** nella RFC 4422 del 2006.

SASL permette di usare diversi meccanismi di sicurezza, tra cui quello indicato come GSSAPI, basato su Kerberos, che offre un livello di sicurezza dei dati.